

第5回

手嶋毅志 (Takeshi Teshima)

質問と回答

告知

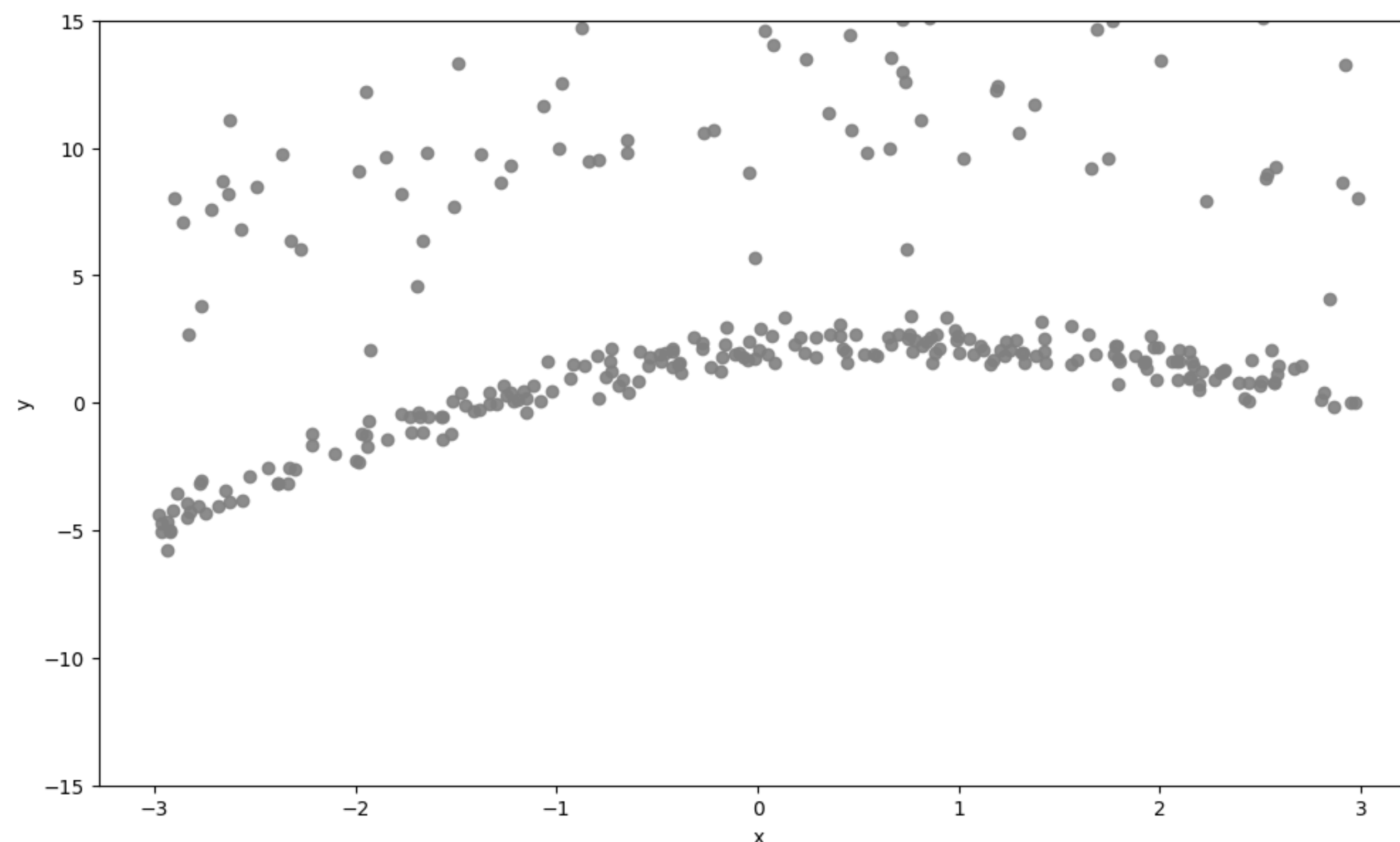
- 小テストは月曜日に公開します

リスク最小化と近似ターゲット（前回の残り）

分位点回帰

次のようなデータの場合の、回帰タスクを考える

- 例) 主な予測対象の集団と、一部の外れた対象がいる
 - 下の集団を重視して予測器を作りたい



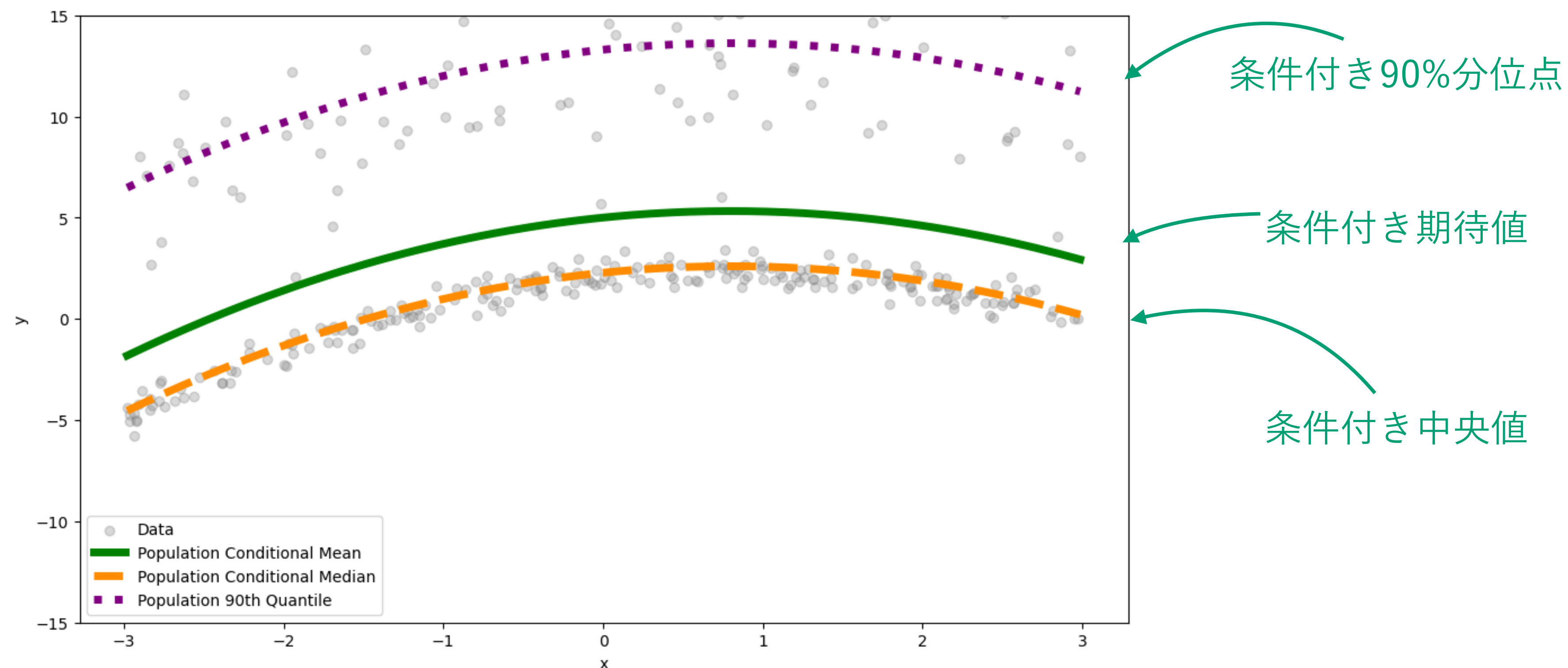
この辺りは重視しない

主な予測対象としたい

分位点回帰

条件付き期待値では，集団から離れた少数に引っ張られやすい

- 条件付き中央値のほうが，条件付き期待値よりも予測器として妥当な場合がある
 - モデルによって条件付き中央値を近似したい

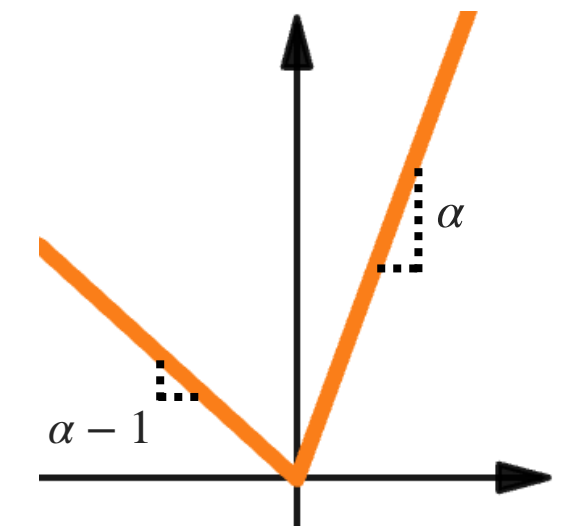


近似ターゲット > 条件付き分位点

近似ターゲットを条件付き分位点にするために、ピンボール損失を使う

- 損失関数としてピンボール損失 (pin-ball loss) を用いるとき,

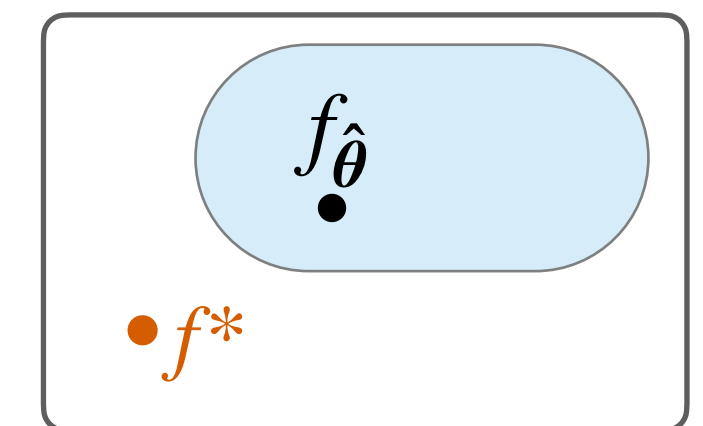
$$\ell_{\alpha}(y, \hat{y}) := \begin{cases} (1 - \alpha) |y - \hat{y}| & (y - \hat{y} < 0) \\ \alpha |y - \hat{y}| & (y - \hat{y} \geq 0) \end{cases}$$



リスクは $R(f) = \mathbb{E}[\ell_{\alpha}(Y, f(X))]$ であり,

- 近似ターゲットは, 条件付き α -分位点 (conditional α -quantile) になる

$$f^*(x) = F^{\leftarrow}(\alpha | x)$$



【補足】ここで $\alpha \in (0,1)$. 【用語】ピンボール損失という名称は、ピンボールを打った軌道のような見た目から、チェックマークの形状のため、チェック損失ともいう。分位点損失 (quantile loss) ともいう。 【参考】Koenker, R., Bassett, G., Jr., 1978. Regression quantiles. *Econometrica* 46, 33–50.

近似ターゲット > 条件付き中央値

近似ターゲットを条件付き中央値にするためには、絶対誤差を使えばよい

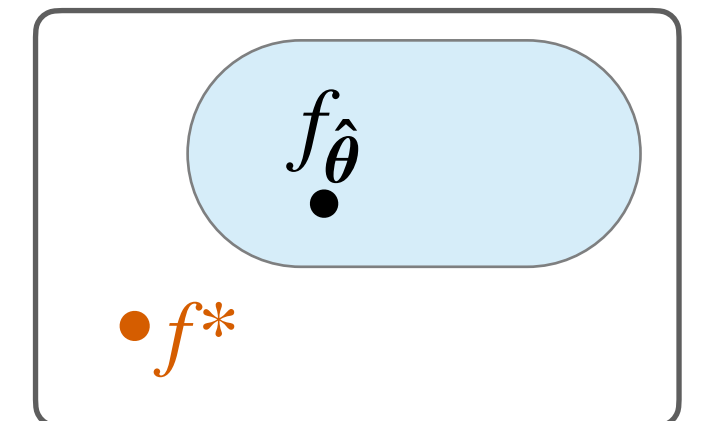
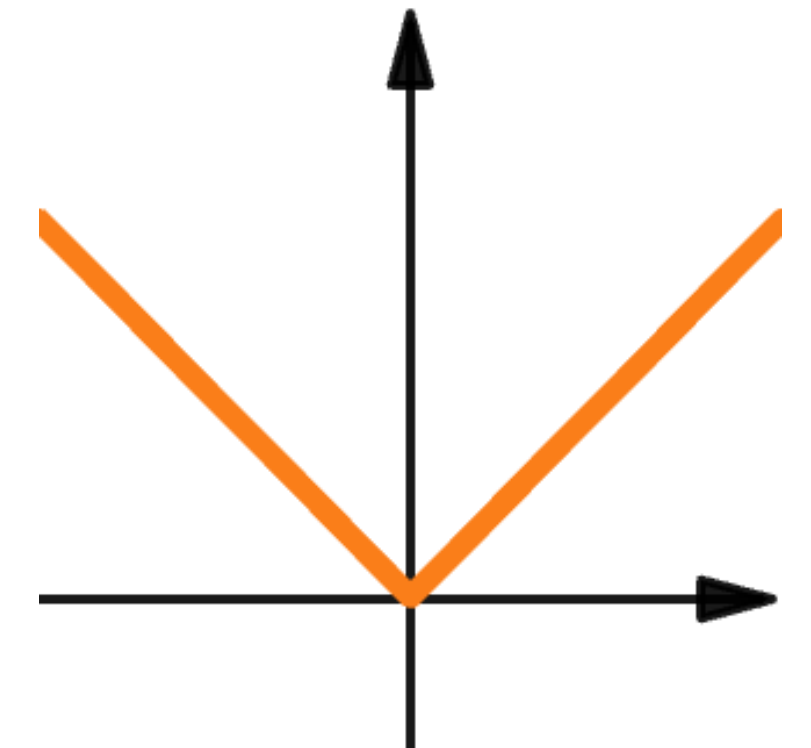
- 絶対誤差 (absolute error) $\ell(y, \hat{y}) = |y - \hat{y}|$ の場合、リスク関数は、

$$R(f) := \mathbb{E} |Y - f(X)|$$

- これは、 $\alpha = 0.5$ のピンボール損失と等価

- この場合の近似ターゲットは、**条件付き中央値関数 (conditional median)**

$$f^*(x) := \inf\{y \in \mathbb{R} \mid \mathbb{P}(Y \leq y \mid x) \geq 0.5\}$$

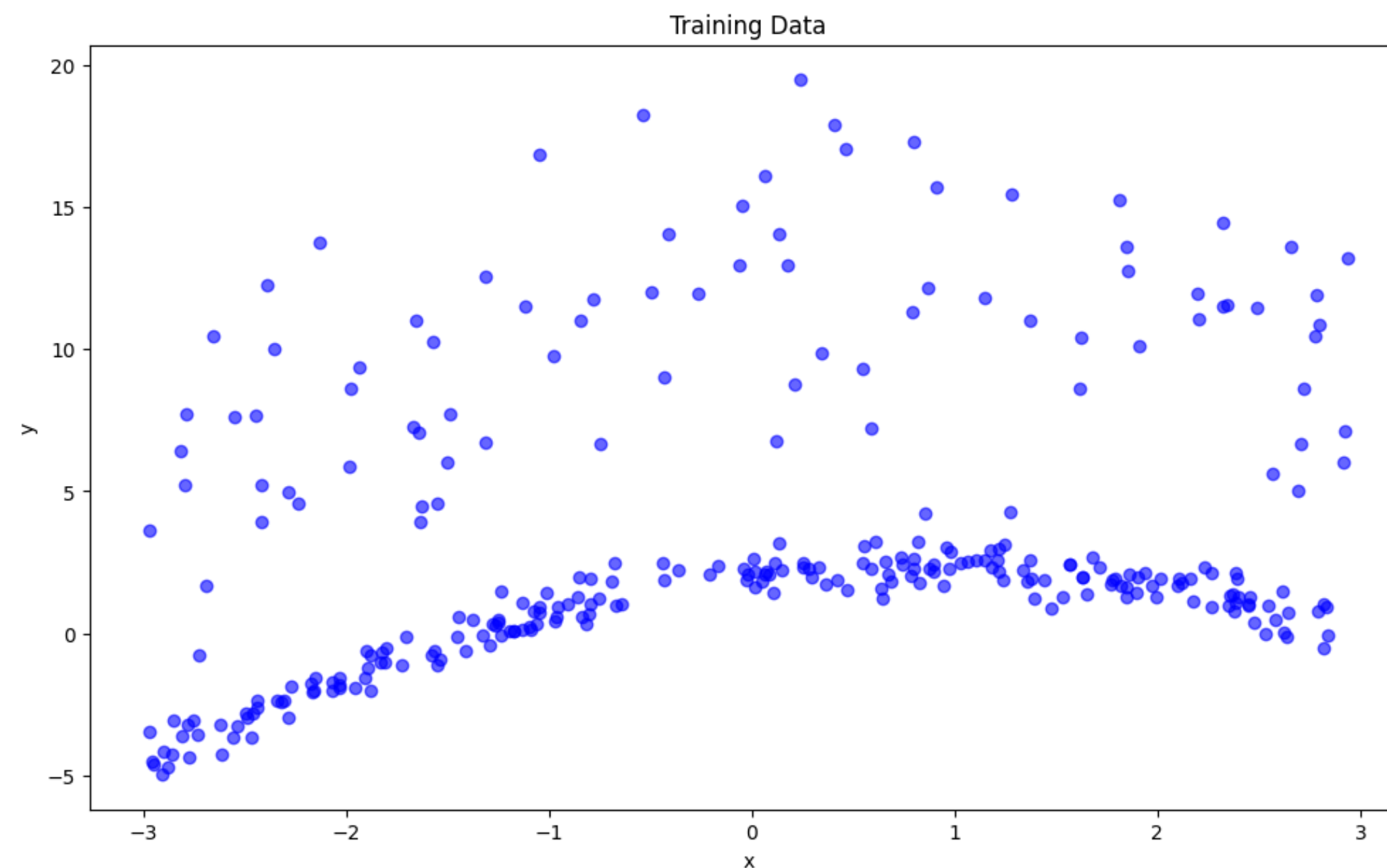


近似ターゲット > 分位点回帰 > 実験

人工データで分位点回帰の実験

■ L2正則化付き最小絶対誤差回帰 $L(\theta) = \sum_{i=1}^n |y_i - f_{\theta}(x_i)| + \lambda \cdot \|\theta\|^2 \rightarrow \text{Minimize}$

- モデルは20次多項式関数, λ は20個ほどの候補から検証損失値により選択



訓練データ

【実験ソースコード】 [番号]_quantile_regression.ipynb

【補足】 最適化の手法の説明は省略するが, 線型計画問題 (linear programming problem) の汎用的な手法を用いて解ける.

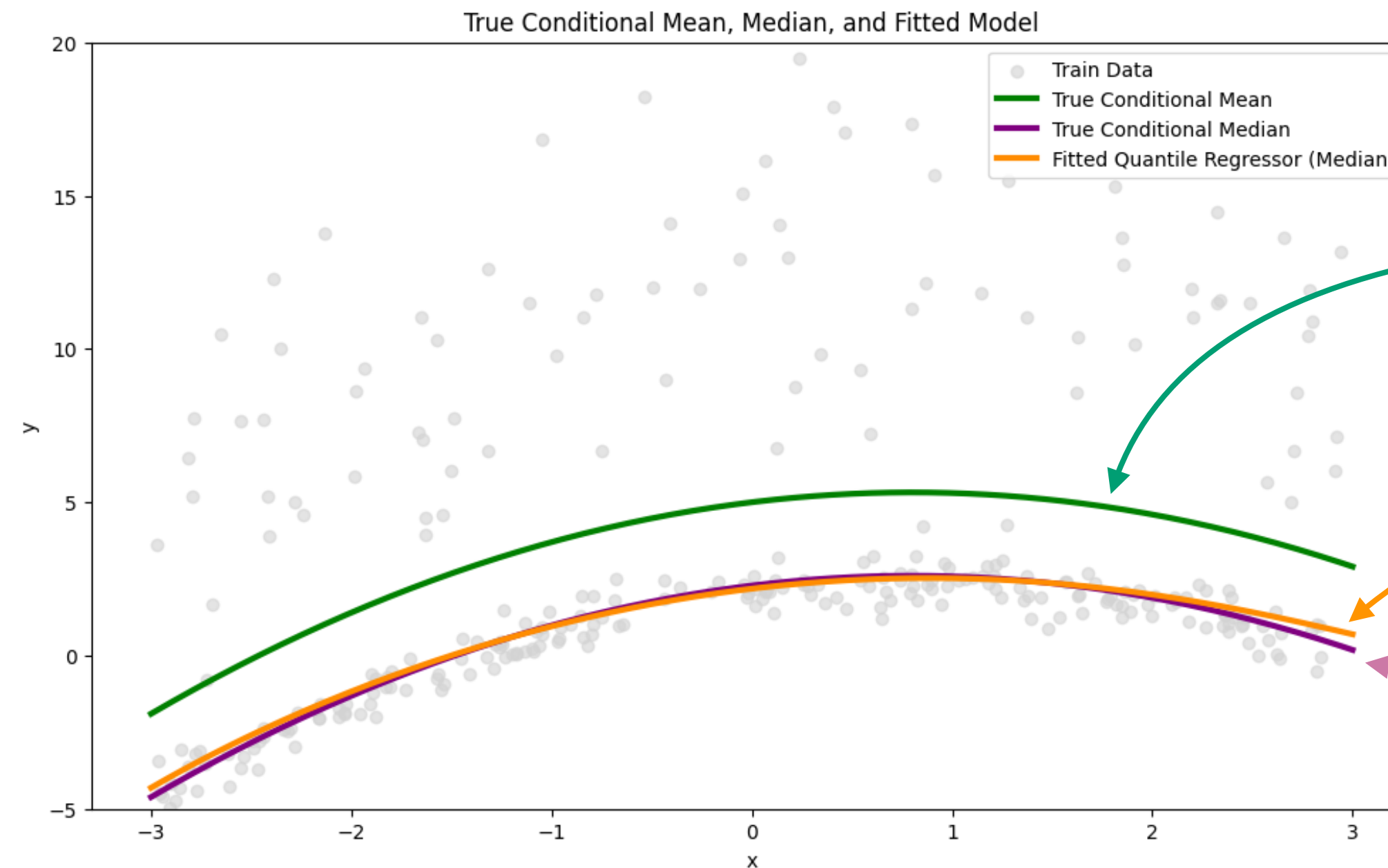
近似ターゲット > 分位点回帰 > 実験

人工データで分位点回帰をした実験の結果

- 訓練済みモデルの近似ターゲットが、条件付き中央値になった様子が見てとれる



訓練データと訓練済みモデル



条件付き期待値

訓練済みモデル

真の条件付き中央値

訓練データと訓練済みモデル

リスク最小化と近似ターゲットまとめ



モデルは、最良の予測器に対する、関数近似器とみることも多い

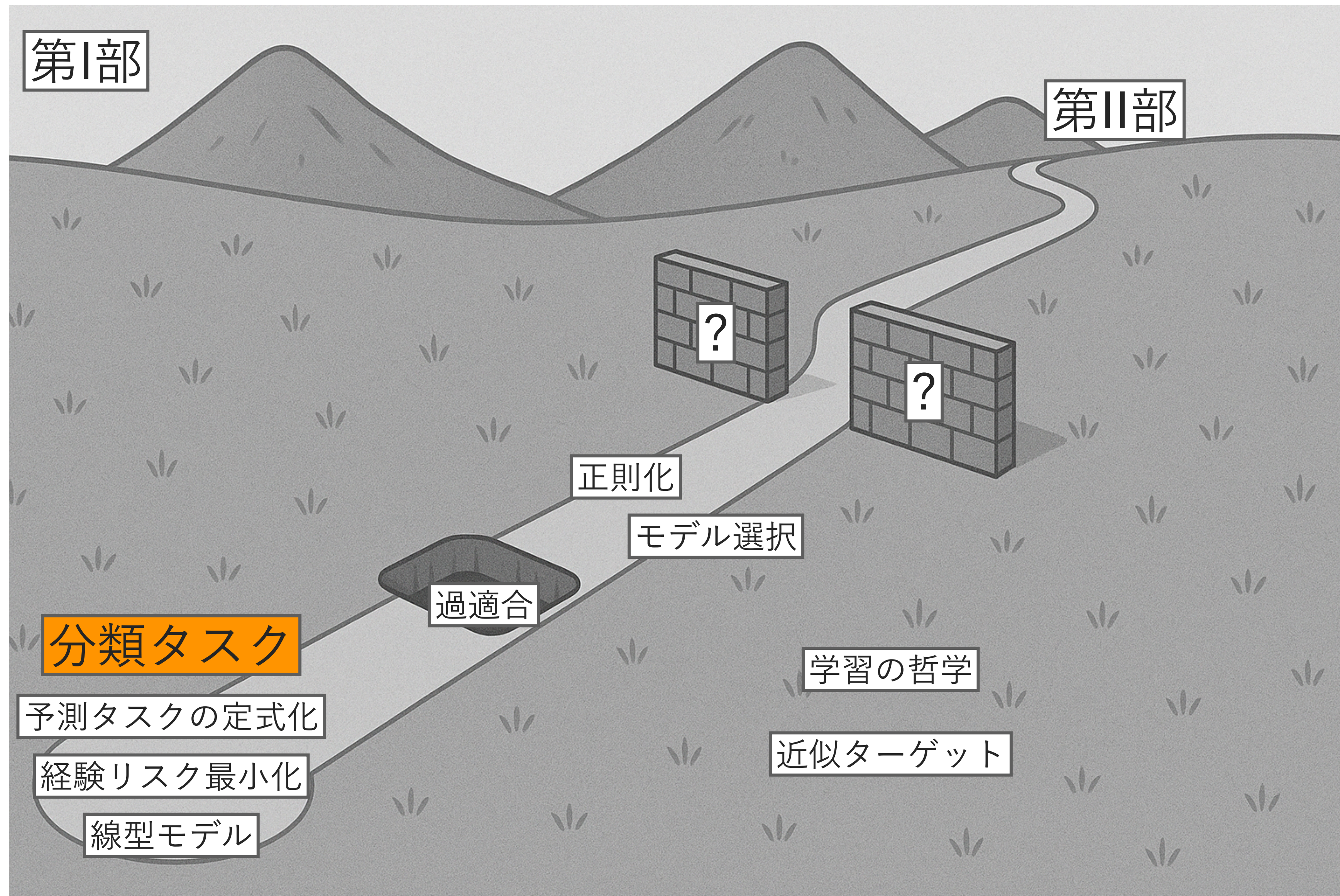
同じ手法の異なる見方

(考えているモデルの中で)
リスクの値ができるだけ小さくなる
ようにパラメーターを調整



(あらゆる予測器の中で)
近似ターゲット
= リスク最小の理想的な予測器
をモデルで近似

ロードマップ



予測系タスク② 分類タスク

分類タスク



ここからは分類タスクと，そのための教師付き学習を説明します

- ここまでは主に回帰タスクを説明してきた
 - ここからは分類タスクのための教師付き学習を説明していく
- 分類タスク
 - 回帰では量的なラベルを予測した（例：3.5, -10.2, ……）
 - 分類では質的なラベルを予測する（例：イヌ, ネコ, ……）
 - 基本的な考え方は同じだが，予測対象が離散的なので少し複雑になる

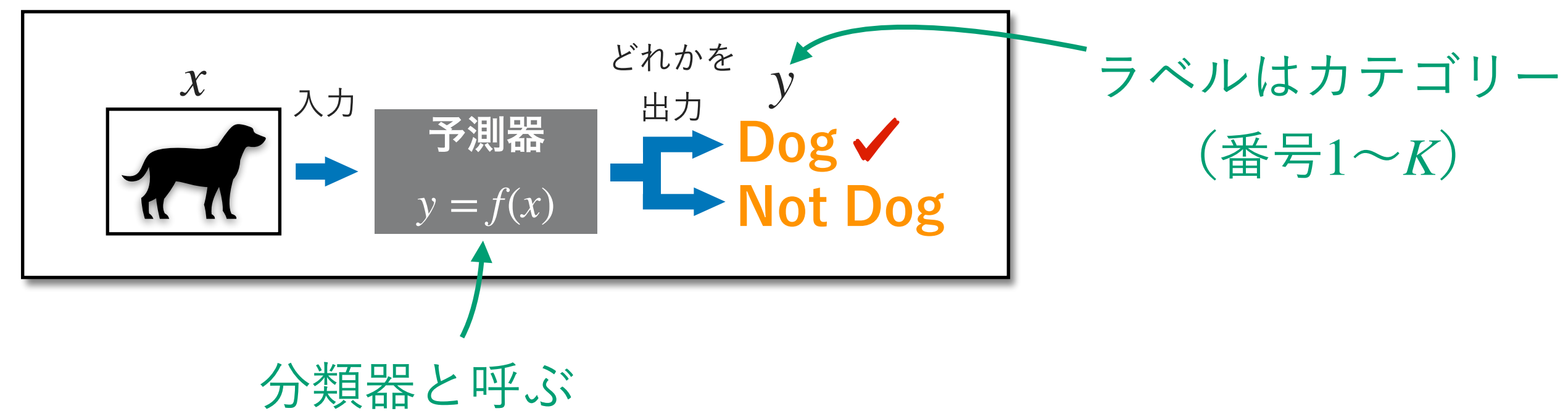
分類タスク

質的なラベルを予測するタスク（ラベルの値は離散的かつ有限個）

■ 分類タスク（classification task）

- ラベルは $\{1, \dots, K\}$ のどれか1つ（ $1, 2, \dots, K$ の大小関係に特別な意味は無い）
- ラベルは、**クラス（class）** や、**カテゴリー（category）** と呼ぶ

■ 未知データも正しく分類できる**分類器（classifier）** を作りたい



分類タスク > 用語 > 2クラス分類

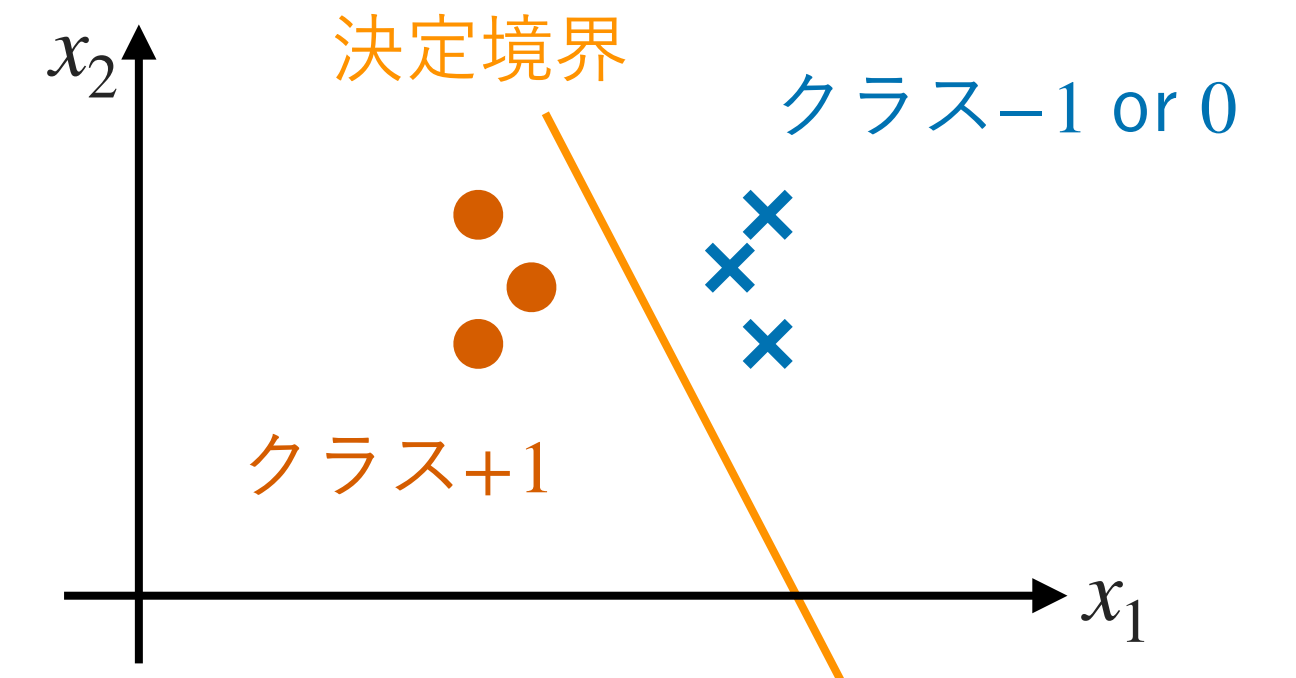
2クラス分類の場合，次のように個別の用語がある

■ 2クラス分類，二値分類 (binary classification)

- クラスは $\{-1, +1\}$ とする場合と， $\{0, 1\}$ とする場合がある

■ 用語

- $y = +1$ や $y = 1$ は正ラベル (positive label) といい，
- $y = -1$ や $y = 0$ は負ラベル (negative label) という
- 正ラベル付きのデータ $(\mathbf{x}, +1)$ ， $(\mathbf{x}, 1)$ は正例 (positive example) といい，
- 負ラベル付きのデータ $(\mathbf{x}, -1)$ ， $(\mathbf{x}, 0)$ は負例 (negative example) という

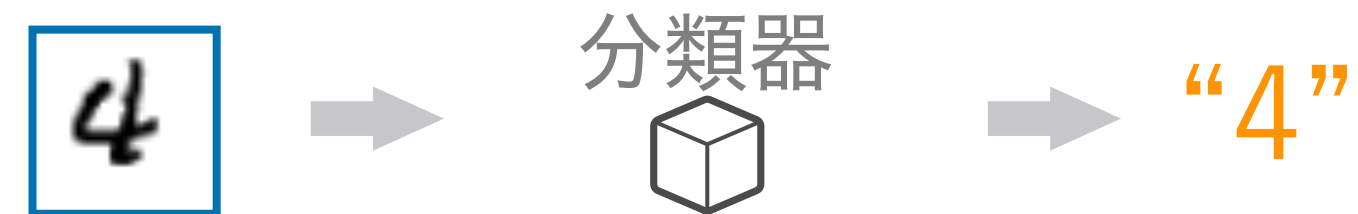


【補足】 特別な用語があることから分かるように，2クラス分類だけでも多くの応用があり，詳しく研究されている。クラスを $\{\pm 1\}$ と $\{0, 1\}$ のどちらにするかは，式が簡単に書けるかどうか 【用語】 3クラス以上の分類タスクは，多クラス分類 (multi-class classification) という。

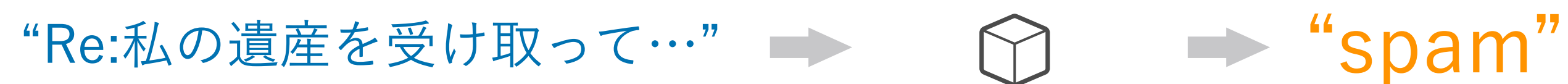
分類タスク＞例

分類タスクの例

- 手書き文字の文字起こし ($\mathcal{X} = \mathbb{R}_{\geq 0}^{20 \times 20}$, $\mathcal{Y} = \{0, 1, 2, \dots\}$)



- メールのスパム判定 ($\mathcal{X} = \text{Vocab}^*$, $\mathcal{Y} = \{\text{spam}, \text{not spam}\}$)



- 翌年の住宅価格が上がるか下がるかを予測 ($\mathcal{X} = \mathbb{R}_{\geq 0}^{12}$, $\mathcal{Y} = \{\uparrow, \downarrow\}$)



分類タスク > リスクの定義

分類の場合の予測誤差は、単に正解・不正解で考えるのが基本

- 分類では損失関数として**0/1-損失 (zero-one loss)** $\ell_{0/1}$ を用いることが多い

$$\ell_{0/1}(y, y') := \mathbf{1}\{y \neq y'\} = \begin{cases} 1 & (y \neq y') \\ 0 & (y = y') \end{cases}$$

正解で損失 0
不正解で損失 1

- この場合のリスクは**誤分類率 (misclassification rate)** や**誤識別率**と呼ぶ

$$R(f) = \mathbb{E} [\ell_{0/1}(Y, f(X))] = \mathbb{P}(Y \neq f(X))$$

不正解の確率

- これが小さい分類器を得ることが基本的な目標

【補足】 0/1-損失の場合のリスクが誤分類の確率に一致することの確認： $R(f) = \mathbb{E} [\ell_{0/1}(Y, f(X))] = \int (\mathbf{1}\{y \neq f(x)\}) \cdot p(x, y) dx dy = \int_{\{y \neq f(x)\}} p(x, y) dx dy = \mathbb{P}(Y \neq f(X))$.

分類タスクの難しさ > ①非数値的な出力をするモデル



1つ目の困難：分類器を作るには、数値的ではないものを出力するモデルが必要

- モデルの数値的な出力を、最終出力である予測ラベルに変換する必要がある
- 回帰では量的なラベルを予測した（例：3.5, -10.2, ……）



- 分類では質的なラベルを予測する（例：イヌ, ネコ, ……）



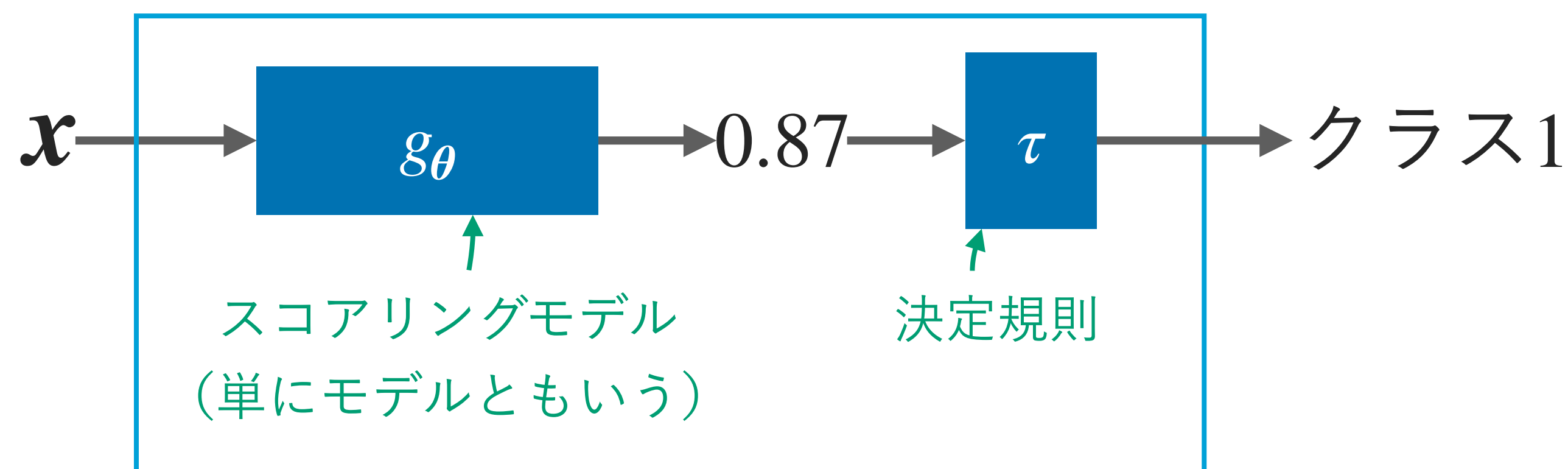
分類タスク＞モデルの作り方

数値的ではないものを出力するためのモデルの基本は、2段階アプローチ

- 離散的なラベルを出力できるモデルを作るために……



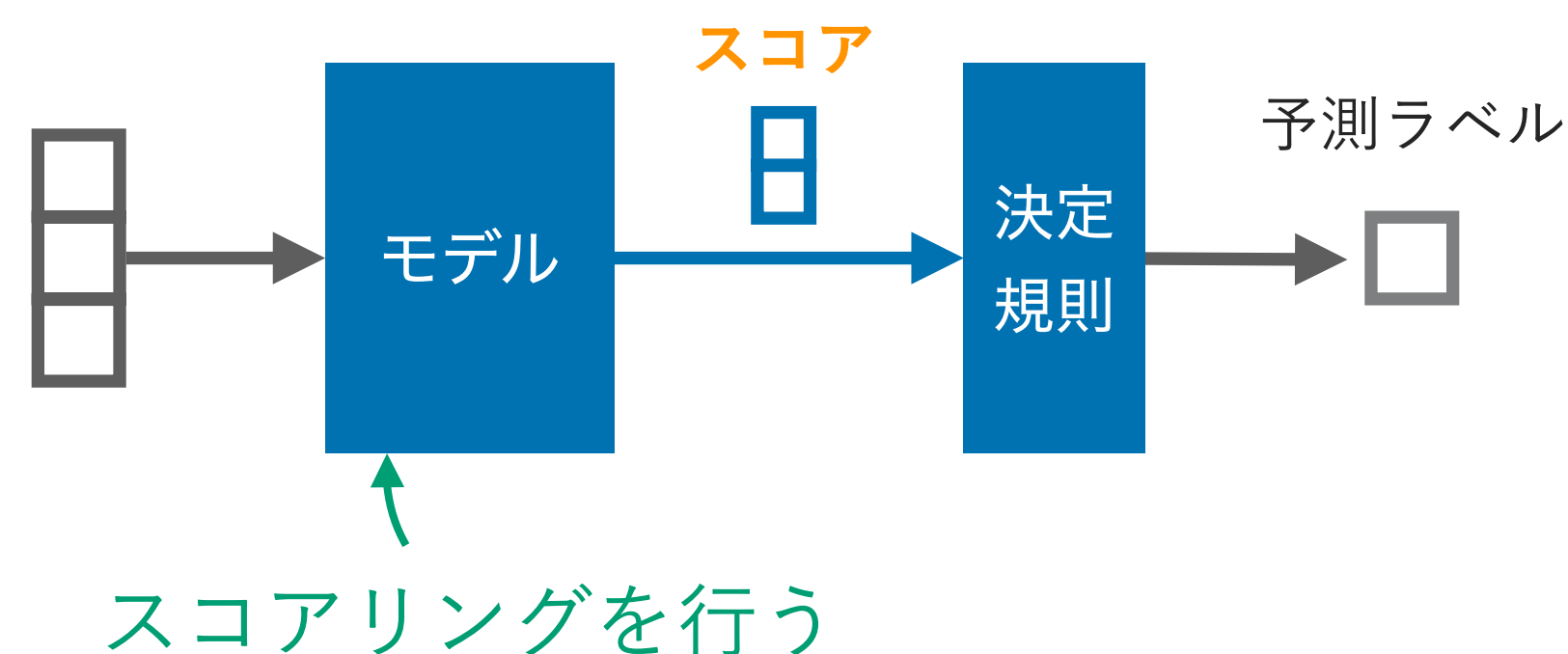
- 数値を出すモデルと、それに応じてラベルを選ぶ決定規則を組み合わせて実現



分類タスク＞モデルの作り方＞スコアリング

モデルからは「どのクラスの可能性が高そうか」を表す数値的スコアを出力

- **スコア (score)** : 内部的な出力値のこと (多次元でもよい)



- 基本的には「どのクラスの可能性が高そうか」を表す数値という気持ちで見る
- 予測器全体とスコアリング部分のいずれも「**モデル**」と呼ぶが、後者を特に指したい場合は「**スコアリングモデル**」という。

【参考】 Sammut, C., & Webb, G. I. (Eds.). (2017). Encyclopedia of Machine Learning and Data Mining. Springer US. p.210

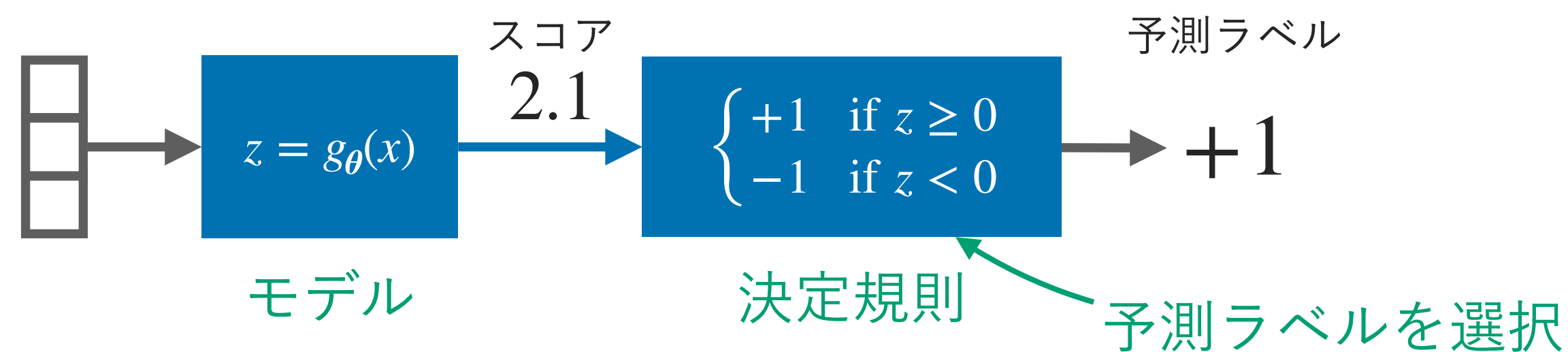
【用語】 なおスコアという用語は多義的に使われるので注意（一般に点数付けを指す）。決定規則も含めてモデルとみなせるが、スコアリングを行う部分までだけをモデルと呼ぶことも多い。また、逆に分類器全体を決定規則と呼ぶこともある。

【注意】 スコアリングモデル＋決定規則という形ではないモデルもある。

分類タスク > モデルの作り方 > 決定規則

モデルが出したスコアを受け取ってラベルを出力する関数を決定規則と呼ぶ

- **決定規則 (decision rule)** : スコアを受け取ってラベルを出力する関数
 - 例) 2クラス分類 $\{-1, +1\}$ で, スコア $z = g_{\theta}(x)$ の符号でラベルを選ぶ



- 上のように基準値（閾値）を超えたか否かでラベルを選ぶ決定規則のことは、特に**閾値処理 (thresholding)** と呼ぶ.

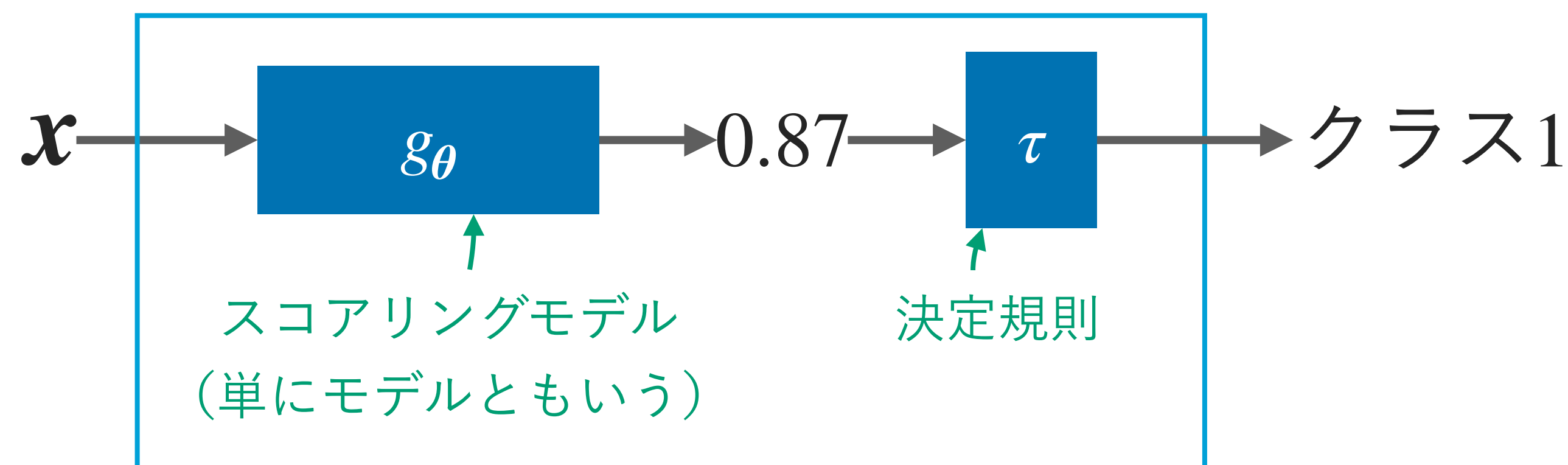
$$\tau(z) := \begin{cases} +1 & (z \geq 0) \\ -1 & (\text{otherwise}) \end{cases}$$

分類タスク＞モデルの作り方



数値的ではないものを出力するモデルの基本は、2段階アプローチ

- モデルからスコアを出して、それを決定規則に入力して予測ラベルを選ぶ



- この講義では主に、スコアリングモデルは g_θ で、決定規則は τ で表記する
 - これらを部品として作った分類器は f_θ で表記（これまでと揃える）

分類タスクの難しさ > ②微分が役立たない損失関数

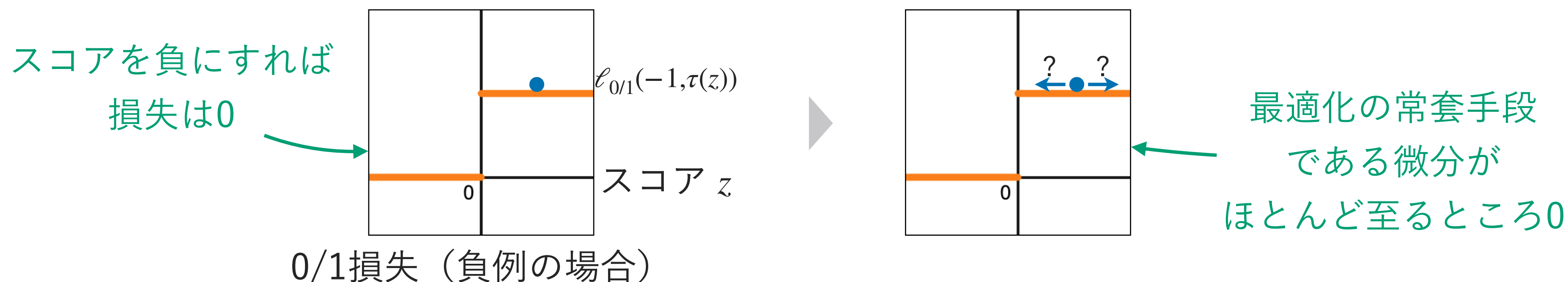


2つ目の困難：損失関数が離散的なので、最適化に不都合が出てくる

- 0/1損失の場合の経験リスクは、訓練標本における誤分類率

$$\hat{R}(g) := \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y_i \neq \tau(g(x_i))\}$$

- しかし、これをそのまま使って目的関数を作ると、実は最適化が困難になる



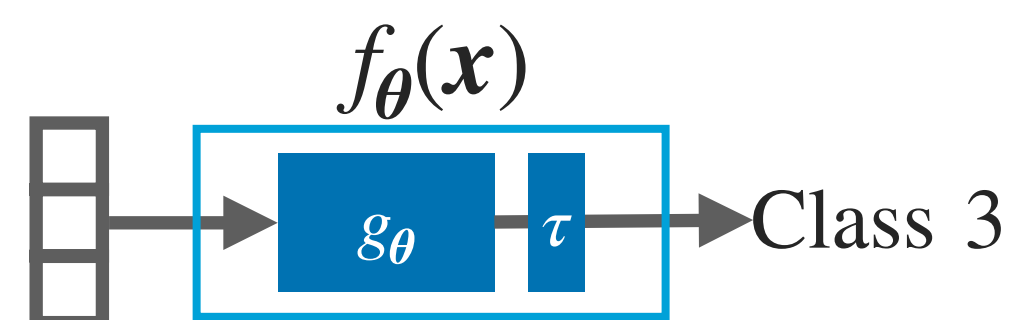
- 0/1損失でリスク定義をすることは問題ないが、肝心のモデルの学習には不適格

分類タスク > 2つのアプローチ

分類タスクの解き方は、大きく2つのアプローチに分かれる

■ 決定論的分類 (deterministic classification)

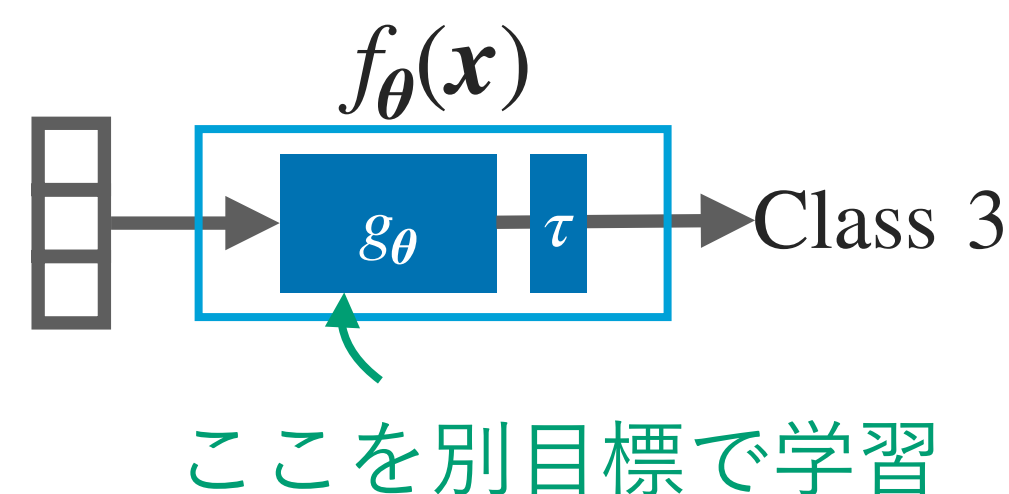
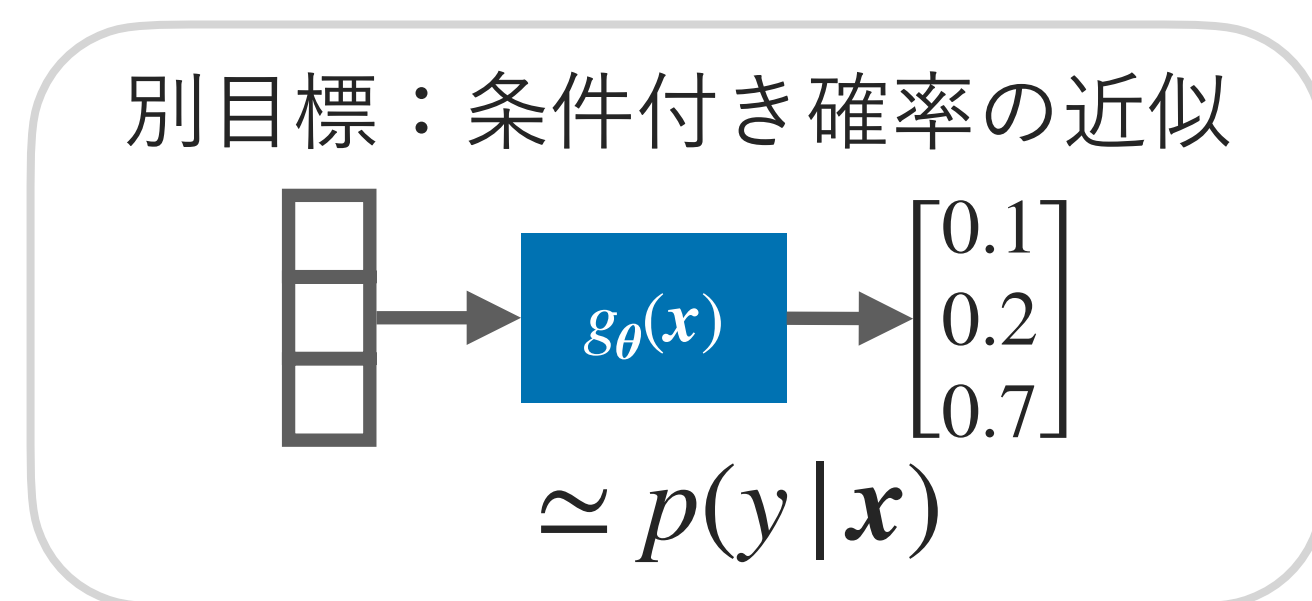
- モデル $f_{\theta}(x)$ のリスク $R(f_{\theta})$ が小さくなることを直接的に目指して学習



$f_{\theta}(x)$ の予測が当たることを目指して訓練
※最適化の難しさは目的関数の
工夫などで乗り越える

■ 確率論的分類 (probabilistic classification)

- スコアモデル $g_{\theta}(x)$ が条件付き確率 $p(y|x)$ をうまく近似するように学習



ゴールはあくまでも
誤識別率が低い $f_{\theta}(x)$ を作ること。
(だが敢えて回り道する)

【用語】決定論的分類は「『確率論的分類』ではないものの総称」と考えると分かりやすい。

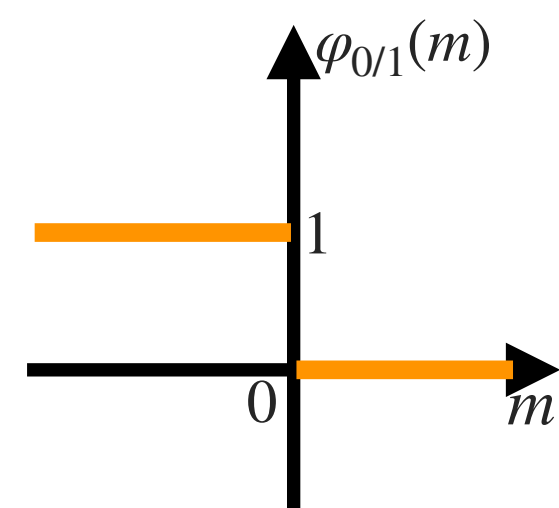
【補足】正確には、決定論的分類手法の中には、必ずしもリスク関数の最小化を学習の第一目標としていないものもあるが、この講義ではあくまでリスクが小さい予測器を作ることをゴールとして扱う。決定論的と呼ばれる手法であっても、データの分布などを考えているものは、本来の意味では確率論的ともいえる。

決定論的分類アプローチ

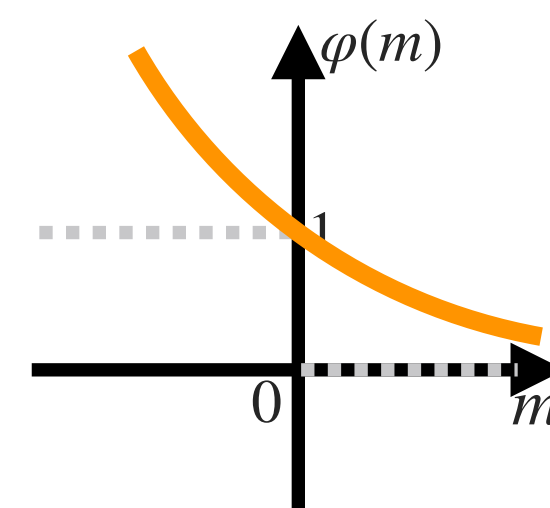


この講義では、決定論的分類アプローチの説明は割愛

- 決定論的アプローチでは、直接的に $R(f)$ が小さいモデルを目指す
- 雰囲気だけ説明しておくとして、最適化が難しい0/1損失の経験リスクの代わりに、最適化しやすくてかつ $R(f)$ が下がる方向に働く代理損失を使うなどする



リスク定義の損失関数
(0/1-損失)



学習用の代理損失

- サポートベクトルマシン (support vector machines, SVM) などはこの系譜.

分類タスクの解き方の分類＞確率論的分類

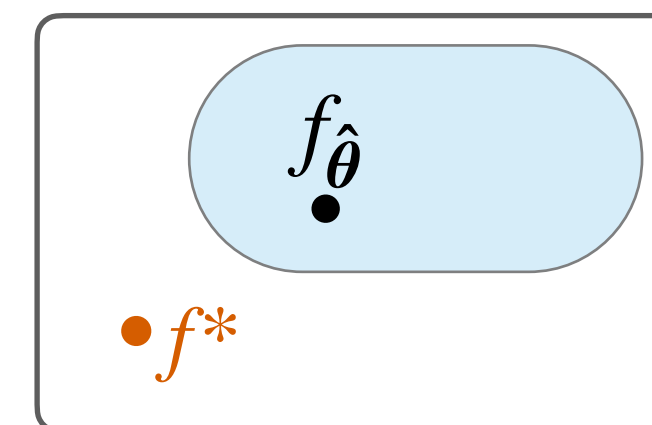
確率論的分類では $p(y|\mathbf{x})$ を一旦推定する. というのも……

- リスク関数を誤分類率 (misclassification rate) にすると,

$$R(f) := \mathbb{E}[\mathbf{1}\{Y \neq f(X)\}]$$

- 近似ターゲットは「 $p(y|\mathbf{x})$ が最も大きいクラスを予測とする」という予測器

$$f^*(\mathbf{x}) = \arg \max_{k \in \{1, \dots, K\}} p(y = k | \mathbf{x})$$



- 最大事後確率分類器 (maximum a-posteriori probability classifier) という
- ベイズ分類器 (Bayes classifier) ともいう

- つまり, 最良の予測器 f^* は $p(y|\mathbf{x})$ を用いて表せることが分かっている

【用語】 ベイズ分類器という用語は, ベイズ統計とは無関係に使われている.

【補足】 複数のクラスが同時に確率最大の場合 (1位タイ) は, それらのいずれかを任意に選ぶことによりタイブレイクすることによっておけばよい.

分類タスク > 費用感応的な損失

0/1損失の代わりに，費用感応的な損失を使いたい場合もある

- 費用感応的な損失（cost-sensitive loss）

$$\ell_{\alpha}(y, \hat{y}) := \alpha_{y, \hat{y}}$$

- どのクラスをどのクラスと取り違えたかによって損失が異なる

		$\hat{y}$		
		Class 1	Class 2	Class 3
y	Class 1	0	0.8	1
	Class 2	5	0	3
	Class 3	0.6	0.2	0

近似ターゲット > 費用感応的な損失

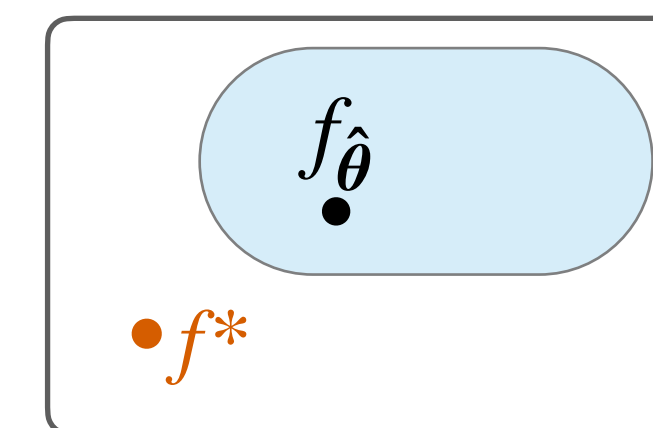
近似ターゲットは，条件付き期待コスト最小のクラスを返す予測器になる

- 費用感応的な損失 $\ell_\alpha(y, \hat{y}) := \alpha_{y, \hat{y}}$ の場合のリスクは

$$R(f) := \mathbb{E}[\ell_\alpha(Y, f(X))]$$

- この場合の近似ターゲットは，「条件付きリスク最小のクラスを返す」予測器

$$f^*(\mathbf{x}) = \arg \min_{k \in \{1, \dots, K\}} \sum_{j=1}^K \alpha_{j,k} \cdot p(y = j | \mathbf{x})$$



条件付きリスク

$$\mathbb{E}[\alpha_{Y,k} | \mathbf{x}] := \sum_{j=1}^K \alpha_{j,k} \cdot p(y = j | \mathbf{x})$$

- この場合も，最良の予測器 f^* は $p(y | \mathbf{x})$ を用いて表せる

【用語】 これもベイズ分類器 (Bayes classifier) という。条件付きリスク (conditional risk) とは $\mathbb{E}[\alpha_{Y,k} | \mathbf{x}]$ のこと。

分類タスクの解き方の分類＞確率論的分類手法

近似ターゲットの見地から、確率論的分類手法では $p(y|\mathbf{x})$ を一旦推定する

■ 分類タスクでの最良の関数

- 0/1損失でのリスク（誤分類率） $\rightarrow f^*(\mathbf{x}) = \arg \max_{1 \leq k \leq K} p(y = k | \mathbf{x})$
- 費用感応的な損失でのリスク $\rightarrow f^*(\mathbf{x}) = \arg \min_{1 \leq k \leq K} \sum_{j=1}^K \alpha_{j,k} \cdot p(y = j | \mathbf{x})$

■ どちらも、まず $p(y|\mathbf{x})$ をモデルで近似しておけば、そこから簡単に作れる

- 確率論的分類では $p(y|\mathbf{x})$ を **クラス事後確率（class posterior）** と呼ぶ

■ どうすればクラス事後確率をモデルで近似できるか？

$\rightarrow$ **近似ターゲットが $p(y|\mathbf{x})$ になるような（都合のよい）損失関数を使う**

分類タスク＞概要のまとめ



分類タスクではラベルが離散的なため、モデルにも最適化にも工夫が必要になる

- 分類タスクの特有の用語がある
- モデルは、スコアリング＋決定規則という形のものが多い
- 決定論的・確率論的という2つのアプローチがある
 - 決定論的アプローチでは、目的関数の工夫などで $R(f)$ が小さいモデルを目指す
 - 確率論的アプローチでは、一旦、条件付き確率 $p(y|x)$ の良い近似を目指す
 - ▶ 回り道ともいえるが、多クラス分類にも自然に対応できることなどのメリットから、近年はこちらが多い

これまでのあらすじ

- 誤分類率 (misclassification rate) リスク

$$R(f) = \mathbb{E} [\mathbf{1}\{Y \neq f(X)\}] = \mathbb{P}(Y \neq f(X))$$

- これに対応する経験リスクは $\frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y_i \neq f(x_i)\}$ だが, 0/1-損失で作る目的関数は最適化が困難という問題がある
- しかし, R を最小化する関数 f が最大事後確率則 $f(x) = \arg \max_y p(y|x)$ であることは分かっているので, $p(y|x)$ を近似することで目的を達成できる

- $p(y|x)$ を近似するには? その方法は今回説明する

- 確率値を出力するモデルの作り方や, その最適化の話も今回説明する

分類タスクへの確率論的アプローチ

確率論的分類アプローチ



ここから「確率的分類アプローチ」を説明

- ここで出てくる重要な考え方：条件付き確率の推定

確率論的分類

誤分類率リスクが小さい分類器を作りたい

- どうやって誤分類率リスクが小さいモデルを作るか？

$$R(f) = \mathbb{E} [\ell_{0/1}(Y, f(X))] = \mathbb{P}(Y \neq f(X))$$

- 0/1損失が入っている最適化は微分が役に立たないので難しい
- ヒント：最適な分類器（近似ターゲット）の形は分かっていた

$$f^*(\mathbf{x}) = \arg \max_{1 \leq k \leq K} p(y = k | \mathbf{x})$$

ここに着目する



確率論的分類

最適な分類器 $f^*(\mathbf{x})$ が、クラス事後確率 $p(y|\mathbf{x})$ を使って表せることに着目

- 最適な分類器は次の形なので、

$$f^*(\mathbf{x}) = \arg \max_{1 \leq k \leq K} p(y = k | \mathbf{x})$$

- モデル $p_\theta(y|\mathbf{x})$ を学習し、決定規則 $\tau(p) = \arg \max_{1 \leq k \leq K} p_k$ で予測する方針をとる

$$f_\theta(\mathbf{x}) = \arg \max_{1 \leq k \leq K} p_\theta(y = k | \mathbf{x})$$

- どうやって条件付き分布のモデル $p_\theta(y|\mathbf{x})$ を作り、 $p(y|\mathbf{x})$ を近似させるか？

確率論的分類＞識別的アプローチ

条件付き分布のモデル $p_{\theta}(y|\mathbf{x})$ を作り, $p_{\theta}(y|\mathbf{x}) \simeq p(y|\mathbf{x})$ となるように学習する

- クラス事後確率 $p(y|\mathbf{x})$ を近似するための方針

1. 条件付き確率の**モデル** $p_{\theta}(y|\mathbf{x})$ を用意
2. **近似ターゲットが $p(y|\mathbf{x})$ になるリスク関数**を選定
3. そのリスクの最小化を目指すための目的関数を設計して, 学習を実行

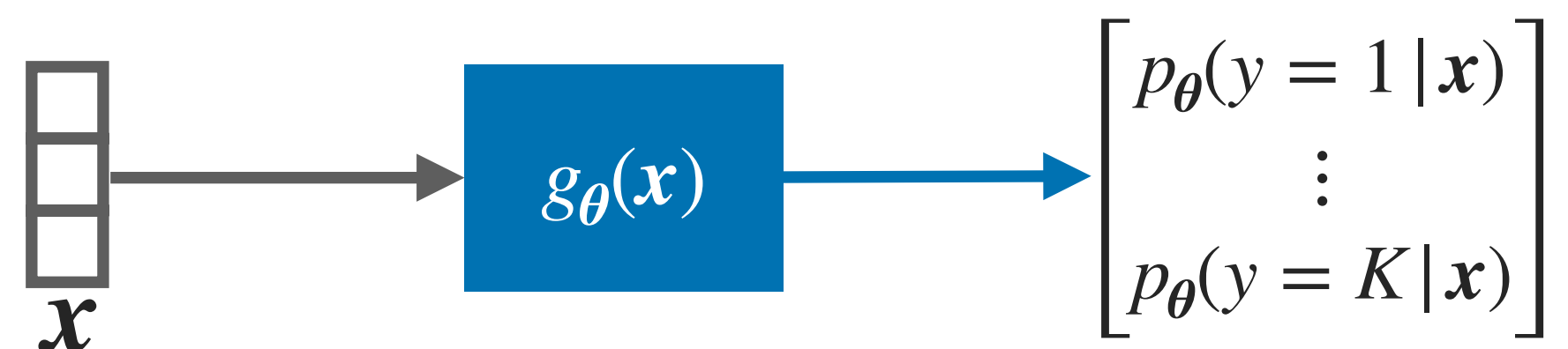
- これは特に, 識別的モデル (discriminative model) アプローチという

- 本節の最後に説明するが, これと対をなす「生成的モデルアプローチ」もある

確率論的分類 > モデル > 要件

確率論的分類には、確率としての要件を満たす値を出力するモデルが必要

- モデルは、 x を受け取って、 $1 \sim K$ の各クラスの確率を出力



- 出力値の要件：**確率値ベクトル (probability/stochastic vector)** であること
 - 各成分は0以上 ($p_k \geq 0$) かつ総和が1である ($\sum_k p_k = 1$) ベクトル p

$$p = \begin{bmatrix} 0 \\ 0.2 \\ 0.7 \\ 0.1 \end{bmatrix}$$

0以上, かつ, 足して1

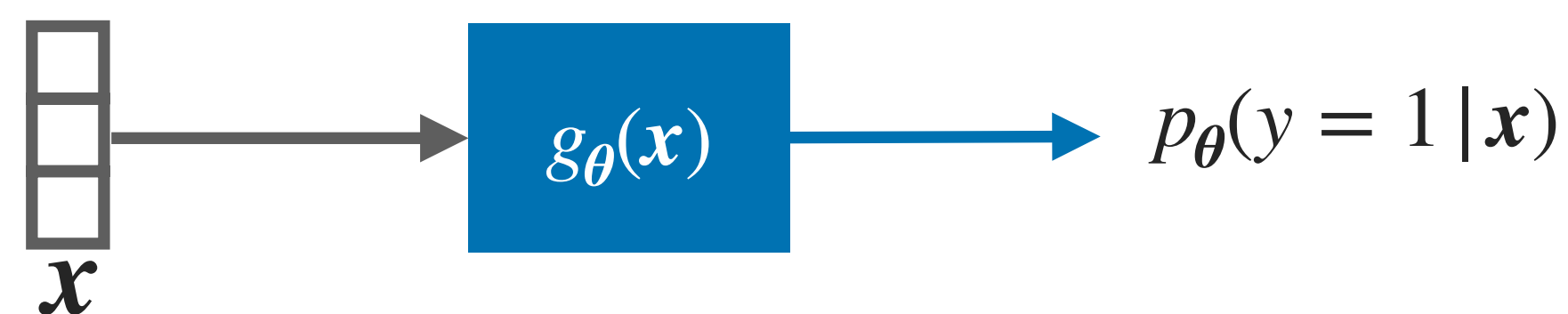
【用語】「確率ベクトル」は確率変数ベクトルを指す用語として使われる場合があるため、この講義では確率値ベクトルと呼び分けることにする。

【用語】 K 個の成分からなる確率値ベクトルを、 K クラスの確率値ベクトル、あるいは K 次元の確率値ベクトルと呼ぶことにする。ただし、これは数学的には $K-1$ 次元の確率単体の元である。

確率論的分類 > モデル > 要件

確率論的分類には、確率としての要件を満たす値を出力するモデルが必要

- 2クラスの場合、モデルは1つの確率値を出力するだけで十分



- 出力値の要件：**確率値 (probability)** であること
 - 0以上1以下の実数 p

$$p = 0.8$$

もう一方のクラスは0.2となる

【補足】2次元の確率値ベクトルを出力させてもよいのだが、単に1つの確率値を出力するだけでも事足りるため、2クラスの場合、通常は、ラベルは $\{0, 1\}$ で扱う。クラス0の事後確率を近似させることにしてもよいが、特に言及がなければ、慣習的にクラス1の事後確率の近似を前提することが多い。

確率論的分類 > モデル > 要件



実際にこれらを満たせるモデルの作り方と最適化の方法は次回以降に解説

- 今回のモデルの説明は、どんなモデルが必要か、という要件の話まで

$$g_{\theta}(x) = 0.125$$

$$g_{\theta}(x) = \begin{bmatrix} 0.3 \\ 0.1 \\ 0.6 \\ 0 \end{bmatrix}$$

確率論的分類＞確率値を近似するための学習方法



確率値を出すモデルは作れたとして、どうやって $p_{\theta}(y|x)$ を学習させるのか？

- リスク関数と近似ターゲットの考え方を活用する！
- リスク関数をうまく選ぶと、その近似ターゲットがクラス事後確率になる
 - そのリスク関数の最小化を目指して学習すれば、結果としてモデルはクラス事後確率の近似を目指していることになる

近似ターゲット > 交差エントロピーの場合

リスクが交差エントロピーのとき、**近似ターゲットはクラス事後確率**

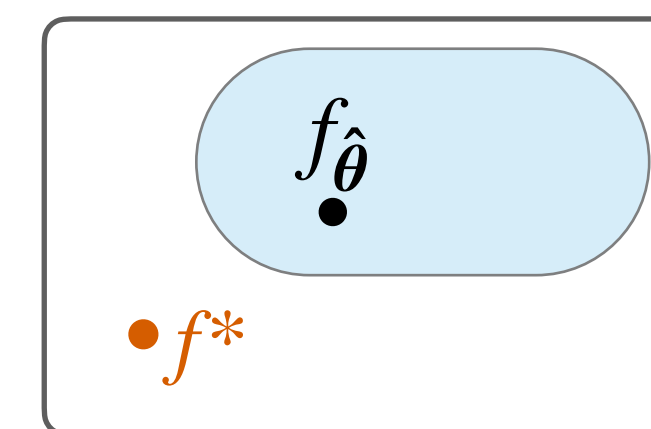
■ 交差エントロピー (cross entropy)

$$R(q) := \mathbb{E}_p[-\log q(Y|X)]$$

- $\ell((x, y), q) = -\log q(y|x)$ は、**交差エントロピー損失 (cross-entropy loss)**

■ この場合の近似ターゲットは、**クラス事後確率 (class-posterior)** になる

$$q^*(y|x) = p(y|x)$$



【注意】ここでの近似ターゲットは、あらゆる関数全体で考えているのではなく、「あらゆる条件付き確率 $q(y|x)$ の全体」の中で考えていることに注意。

【補足】ここで、損失関数は「 $f(x)$ と y の誤差を測る関数」から一般化して、データ点 (x, y) と関数 q に対して値を返す関数 $\ell((x, y), q)$ であればよいものとした。

【補足】近似ターゲットが条件付き確率になるようなリスクは他にもある。【用語】交差エントロピー損失は、対数損失 (log loss) と呼ぶこともある。

【用語】一部では、交差エントロピー損失という名前で、後述のロジスティック関数やソフトマックス関数を合成した負の対数尤度を指す場合がある。

交差エントロピー＞用語

交差エントロピーの経験リスク最小化は、最尤法と呼ばれる

■ 交差エントロピーの経験リスク

$$\frac{1}{n} \sum_{i=1}^n (-\log p_{\theta}(y_i | x_i))$$

- $-\log p_{\theta}(y | x)$ を、データ (x, y) の **負の対数尤度** (negative log likelihood) という
- この場合の経験リスク最小化は **最尤法** (maximum likelihood) という

■ この負の対数尤度をもとにして、学習の目的関数を作る（最尤法ベースの学習）

【補足】「最尤法ベースの」と言っているのは、実際は正則化なども加えることにより、最尤法そのものではないことが多いため。

交差エントロピー > 性質

交差エントロピーは、KLダイバージェンスという量に関係

■ KLダイバージェンス (Kullback-Leibler divergence)

$$\text{KL}(p\|q) := \int p(x, y) \log \left(\frac{p(x, y)}{q(x, y)} \right) dx dy$$

■ KLダイバージェンスの最小化は交差エントロピーの最小化と等価

$$\text{KL}(p\|q) = \underbrace{\mathbb{E}_p[-\log q(Y|X)]}_{\text{cross-entropy}} + \underbrace{\mathbb{E}_p[\log p(Y|X)]}_{\text{constant}}$$

定数なので最適化では無関係

【補足】 条件付き確率 $p(y|x)$ のみをモデル化している場合、特に断らない限り、同時密度のモデルとしては真の入力分布 $p(x)$ と組み合わせた $q(x, y) = q(y|x)p(x)$ を用いることにする。 【用語】 日本語では「カルバック・ライブラー・ダイバージェンス」と表記することが多い。また、幅広い分野で使われる量のため、相対エントロピー (relative entropy)、KL情報量などの別名も多い。

KLダイバージェンス

確率分布間の差異を計るための定量的な尺度として多用される

■ p から q へのKLダイバージェンス

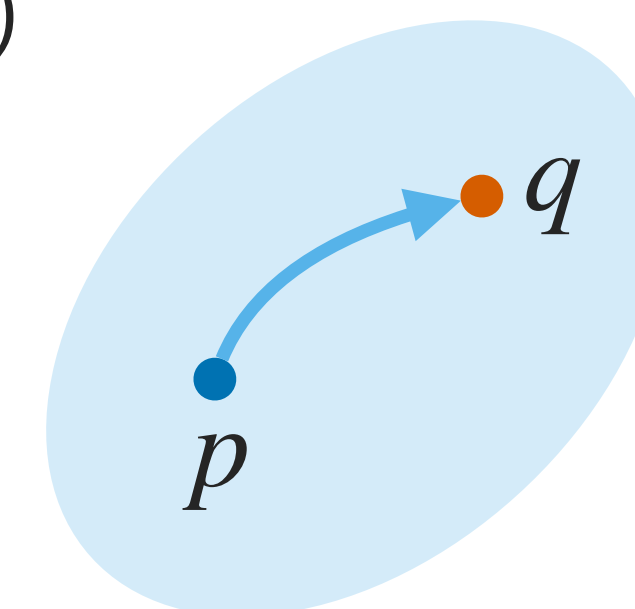
$$\text{KL}(p\|q) := \int p(x, y) \log \left(\frac{p(x, y)}{q(x, y)} \right) dx dy$$

- 性質① 向きがある量 (p から見て q がどれだけ離れているかを表す)

$$\text{KL}(p\|q) \neq \text{KL}(q\|p)$$

- 性質② 値は0以上で、かつ、値が0になるのは $p = q$ のときに限る

$$\text{KL}(p\|q) \geq 0 \quad \text{かつ} \quad \text{KL}(p\|q) = 0 \Leftrightarrow p = q$$



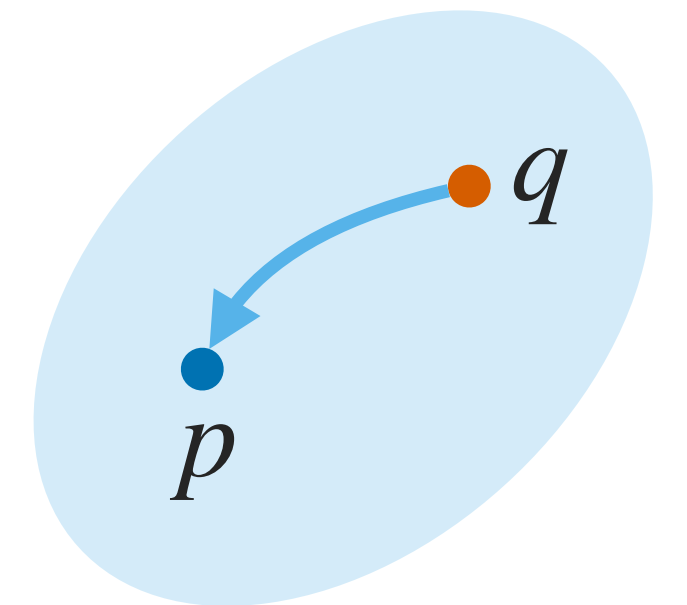
【注意】 上の性質からわかるように、これは数学的には距離の公理を満たさない。 【補足】 上記の等号成立条件は、正確には p -a.s. での成立。

KLダイバージェンス以外の選択肢



なぜKLダイバージェンス（から派生した交差エントロピー）で学習するのか？

- KLダイバージェンスの他にも分布間の相違度指標は色々ある。
 - 逆KLダイバージェンス（reverse KL divergence） $KL(p||q) \leftrightarrow KL(q||p)$
 - α -ダイバージェンス， β -ダイバージェンス， γ -ダイバージェンス
 - ヘリンジャー距離（Hellinger distance）
 - ワッシャースタイン距離（Wasserstein distance）， ……
- どれを使うのが良いか？結局はケースバイケース。



【補足】 KLダイバージェンスは情報理論など至るところに現れる「由緒正しい」分布間相違度なので好まれて使われるが、相違度指標には一つ一つ特性があり、当然ながら、KLダイバージェンスにもそれなりの「クセ」がある。

確率論的分類アプローチ > まとめ



分類タスクへの確率論的なモデルによるアプローチを解説

■ 「確率的分類アプローチ」を説明

- 誤分類率を最小にする分類器は、クラス事後確率 $p(y|\mathbf{x})$ を使って表される
- そこで、手持ちのデータからクラス事後確率 $p(y|\mathbf{x})$ を推定することにした
- クラス事後確率の推定は、交差エントロピーをリスクとする学習で行う
 - ▶ これは、最尤法ベースの学習という
 - ▶ 最尤法ベースの学習で $p(y|\mathbf{x})$ を近似できるのは、KLダイバージェンス由来

確率論的分類 > 補足



$p(y|x)$ の推定をする確率論的分類において、大きく2種類のアプローチがある

	モデリングの対象	考え方の違い
識別的モデルアプローチ	条件付き分布 $p_{\theta}(y x)$	直接的に分類に使う部分だけを学習する
生成的モデルアプローチ	同時分布 $p_{\theta}(x, y)$	データの生成規則の全体を学習する

■ どちらが良いかはケースバイケース

- 現在はまず識別的アプローチを考えることが多いと思われる
- そのため、この講義では分類タスクへの生成的モデルアプローチは割愛

【注意】 生成的モデルといっても、生成タスクのために使えるとは限らない。「生成タスクのための」生成モデルとは関連はあれど異なるものと考えたほうがよい。

確率論的分類＞補足＞生成的モデリング

この講義では説明しないが、次のような方針で $p_{\theta}(y|\mathbf{x})$ を学習する方法もある

■ 生成的モデル (generative model) アプローチ

1. まず、**同時確率のモデル** $p_{\theta}(\mathbf{x}, y)$ を準備する

▶ $(p_{\theta}(\mathbf{x}, y) = p_{\theta}(\mathbf{x}|y) \cdot p_{\theta}(y))$ のように分けてモデル化することも多い)

2. これを、何らかの方法で $p_{\theta}(\mathbf{x}, y) \simeq p(\mathbf{x}, y)$ となるように学習する

▶ $(p_{\theta}(\mathbf{x}|y) \simeq p(\mathbf{x}|y) \text{ と } p_{\theta}(y) \simeq p(y))$ のように分けて学習することも多い)

3. クラス事後確率は、 $p_{\theta}(\mathbf{x}, y)$ から、 $p_{\theta}(y|\mathbf{x}) = \frac{p_{\theta}(\mathbf{x}, y)}{\int p_{\theta}(\mathbf{x}, y) dy}$ として計算

【用語】 生成的という修辭の由来はおおよそ次のとおり。 $p(\mathbf{x}|y)$ はクラスごとの特徴量の生成分布であり、例えば「3という数字に対する手書き文字の画像が従う生成分布」などが相当する。これをモデル化して推定することは、分類という目的のために、一旦「データの生成規則を丸ごと学習する」ようなものである。

2クラスの確率論的分類

2クラスの確率論的分類（確率論的二値分類）



確率論的分類を，まず2クラス分類の場合から説明

- 誤分類率 $R(f) = \mathbb{E} [\ell_{0/1}(Y, f(X))] = \mathbb{P}(Y \neq f(X))$ に対して最適な予測器の形は，

$$f^*(\mathbf{x}) = \arg \max_{1 \leq k \leq K} p(y = k | \mathbf{x})$$

- そこで，モデルで $p(y | \mathbf{x})$ を近似するよう学習し，その値が最大なクラスを出力

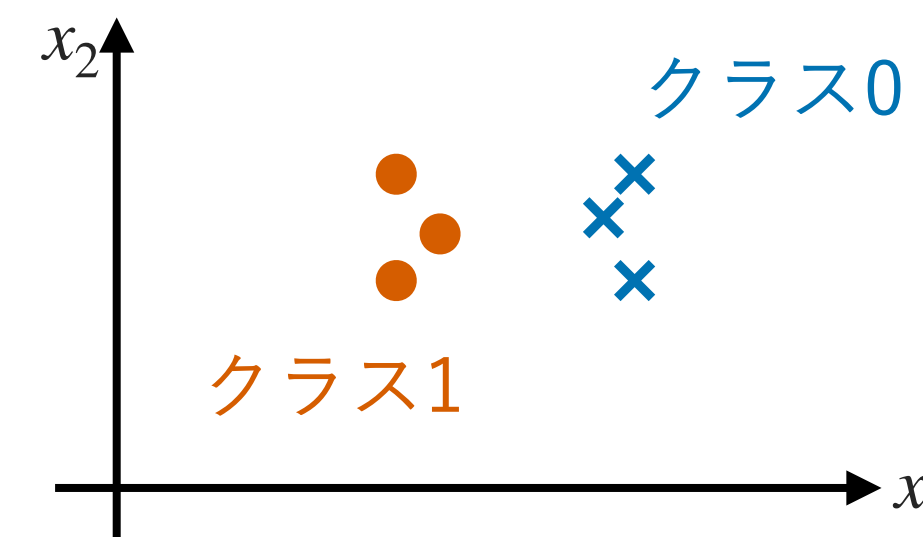
$$f_{\theta}(\mathbf{x}) = \arg \max_{1 \leq k \leq K} p_{\theta}(y = k | \mathbf{x})$$

- 2クラスの場合は，もう少し単純になる

確率論的二値分類 > モデル

確率論的二値分類では、モデルがいくらか単純になる

- 前提として通常、確率論的二値分類では、ラベルは $\{0, 1\}$ とする.
- モデルは単純になる： $p(y = 1 | \mathbf{x})$ だけ近似すればよい



$$\begin{bmatrix} p(y = 1 | \mathbf{x}) \\ p(y = 0 | \mathbf{x}) \end{bmatrix} \rightarrow p(y = 1 | \mathbf{x})$$

- 決定規則も単純になる：最大事後確率則は、0.5での閾値処理で表せる

$$\tau(\mathbf{p}) = \arg \max_{1 \leq k \leq K} p_k \rightarrow \tau(p) = \begin{cases} 1 & (p \geq 0.5) \\ 0 & (p < 0.5) \end{cases}$$

【補足】 負クラスの事後確率は $p(y = 0 | \mathbf{x}) = 1 - p(y = 1 | \mathbf{x})$ と考えればよい.

確率論的二値分類 > 分布近似のためのリスク関数

確率論的二値分類では、交差エントロピー損失も、特有の表式がある

- ラベルが $\{0,1\}$ で、モデルが $g(x) := q(y = 1 | x)$ なら、交差エントロピー損失は、

$$\ell((x, y), g) = -\log q(y | x) = -y \cdot \log g(x) - (1 - y) \cdot \log(1 - g(x))$$

となり、リスク関数は二値交差エントロピー（binary cross entropy）となる。

$$R(g) := \mathbb{E}[-Y \log g(X) - (1 - Y) \log(1 - g(X))]$$

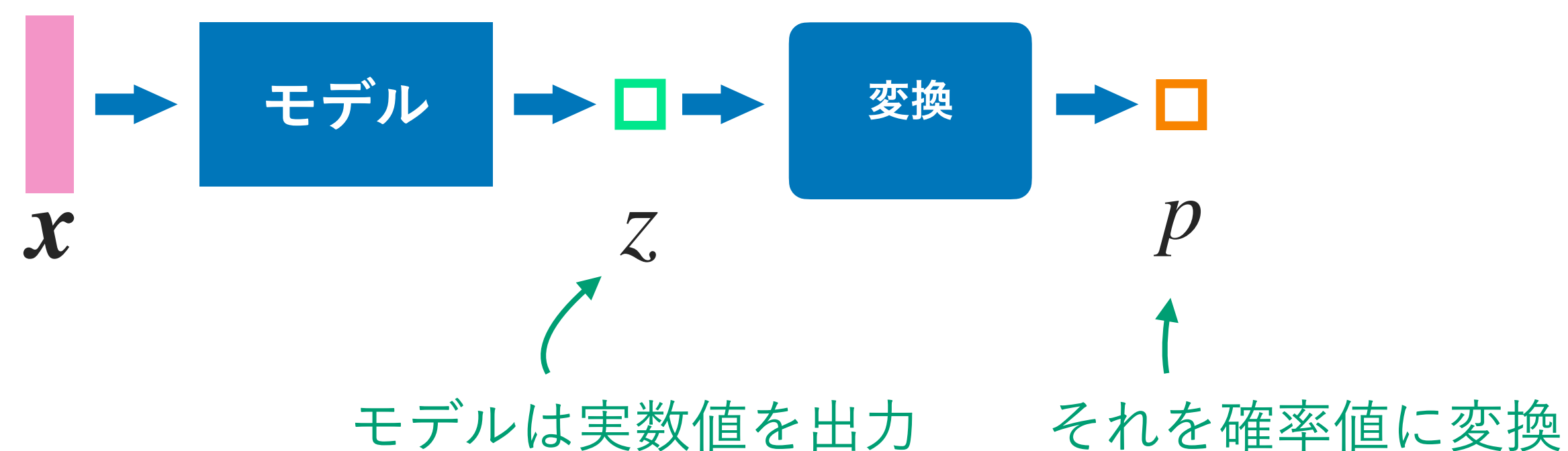
- 単に、交差エントロピーを、2クラスの場合の記号のもとで表しただけのこと
 - このリスク関数での近似ターゲットは、もちろん $g^*(x) = p(y = 1 | x)$ である

【注意】ここでの近似ターゲットは、あらゆる関数ではなく、「出力が確率値であるような、あらゆる関数」の中での最小化元である。

確率論的二値分類＞確率値を出力するモデルの作り方

出力が確率値になるモデル $g_{\theta}(x)$ を用意する必要がある

- 線型モデルなどは、 $[0,1]$ の範囲外の実数も出力してしまう
 - そのままでは確率のモデルとして不適格
- 対策：出力が $[0,1]$ の範囲に収まるように、出力値の変換を行う関数を合成

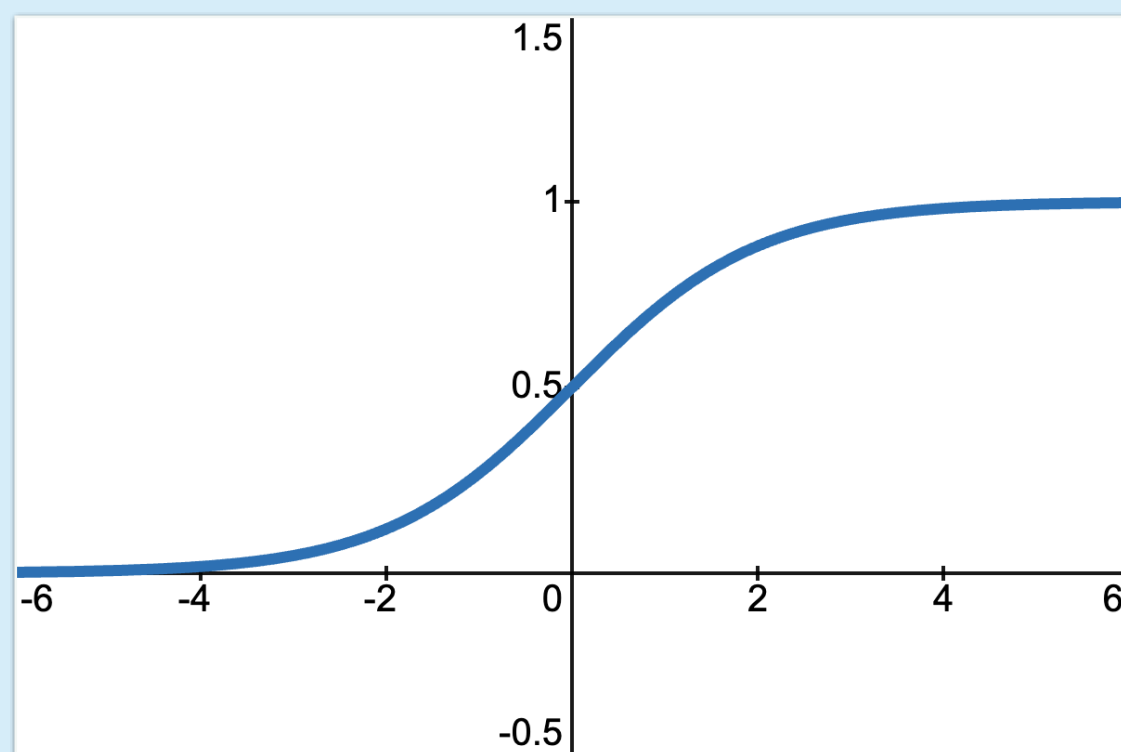


確率論的二値分類＞確率値を出力するモデルの作り方

出力値を[0,1]の範囲に収めたい場合，次のような変換を用いる

ロジスティック関数（シグモイド関数）

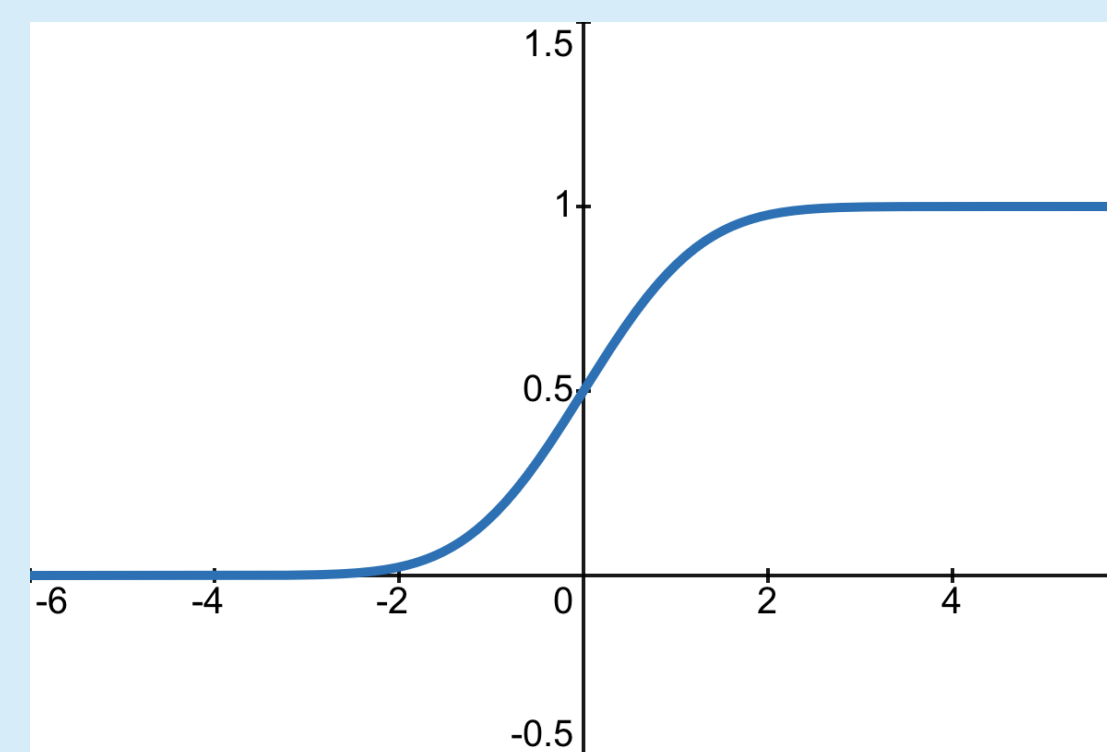
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- 非常によく使われる

標準正規分布の累積分布関数

$$\Phi(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^z e^{-\frac{t^2}{2}} dt$$



- 機械学習では頻繁には使われない

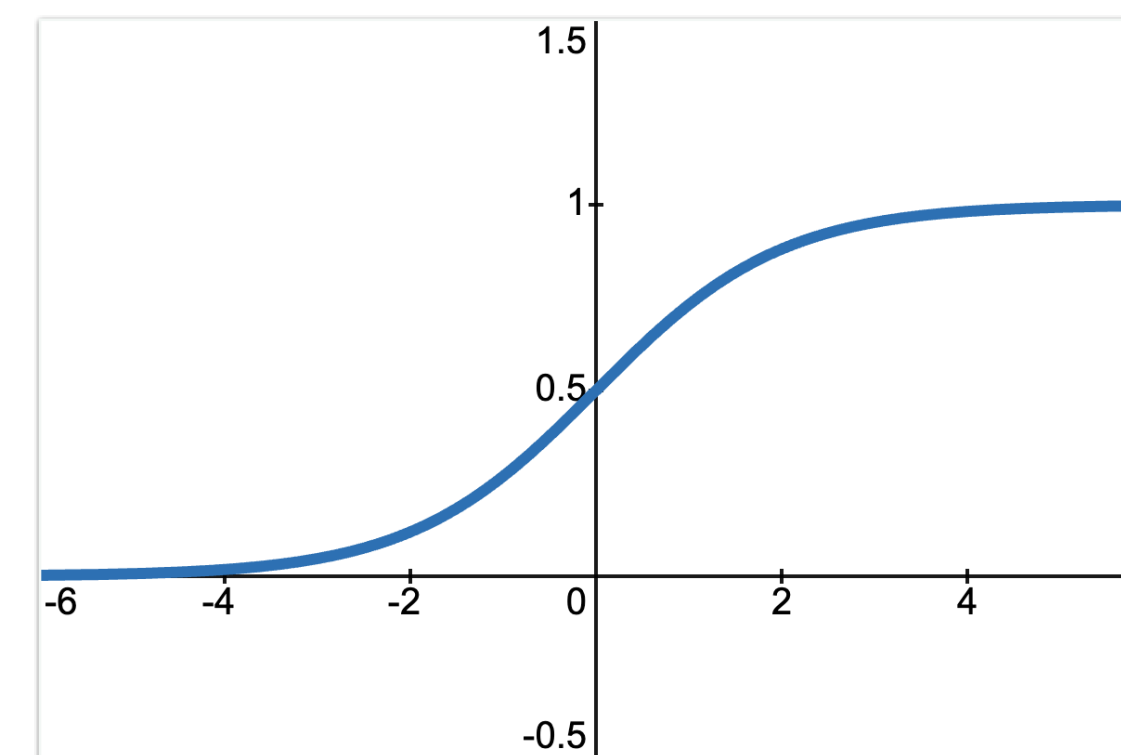
【用語】一般化線型モデルなどの文脈では，これらの関数は逆リンク関数（inverse link function）や平均関数（mean function）と呼ばれることがある．機械学習でシグモイド関数といえば，通常は左の（標準）ロジスティック関数を指す．広義にはシグモイドはS字カーブの形状（ギリシャ文字シグマの語末形ςの形状）を持つ関数全般を指す（右の累積分布関数も含む）．

確率論的二値分類＞モデル＞ロジスティック関数

ロジスティック関数がどんな関数か観察しておこう

■ ロジスティック関数（シグモイド関数）

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- 分母は 1 より真に大きい正の値 → 値域は (0,1)（つまり確率値を出す関数）
- z が大きく負の値 → e^{-z} が大きく正の値 → 分母が大きくなる → $\sigma(z) \simeq 0$
- z が 0 付近 → e^{-z} が 1 付近になる → 分母が 2 付近になる → $\sigma(z) \simeq 0.5$
- z が大きく正の値 → e^{-z} が 0 に近づく → 分母が 1 に近づく → $\sigma(z) \simeq 1$

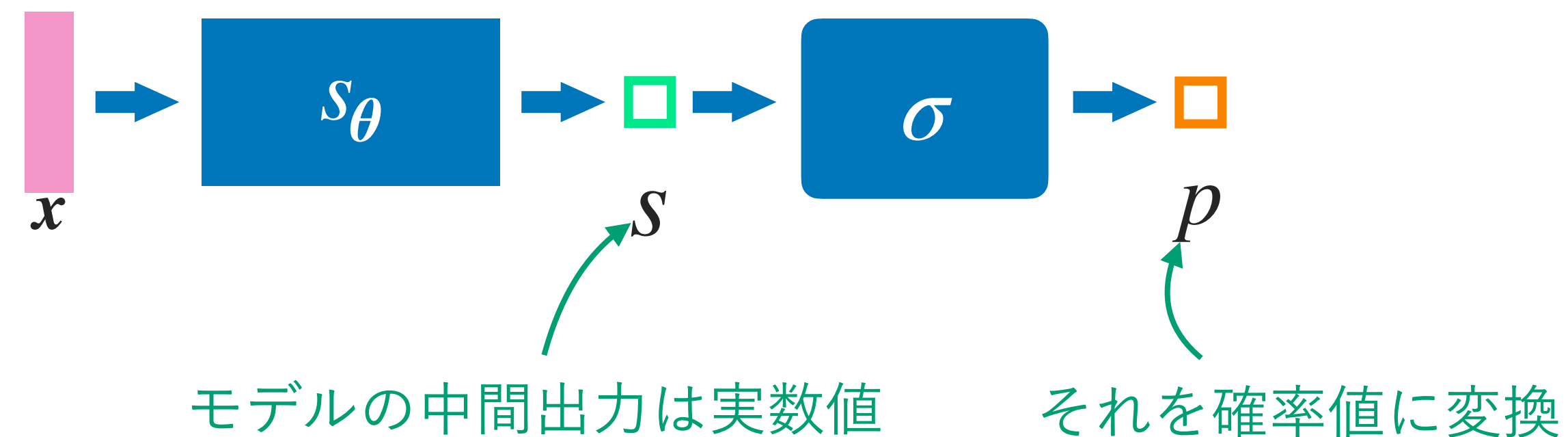
■ 実数の中間的スコアを確率値に潰すための関数として使われるものの一つ

確率論的二値分類＞モデル＞ロジスティック関数

モデルの中間的な出力に，ロジスティック関数を合成して確率値のモデル化

- 確率値を出力するモデル

$$g_{\theta}(x) = \sigma(s_{\theta}(x)) = \frac{1}{1 + e^{-s_{\theta}(x)}}$$

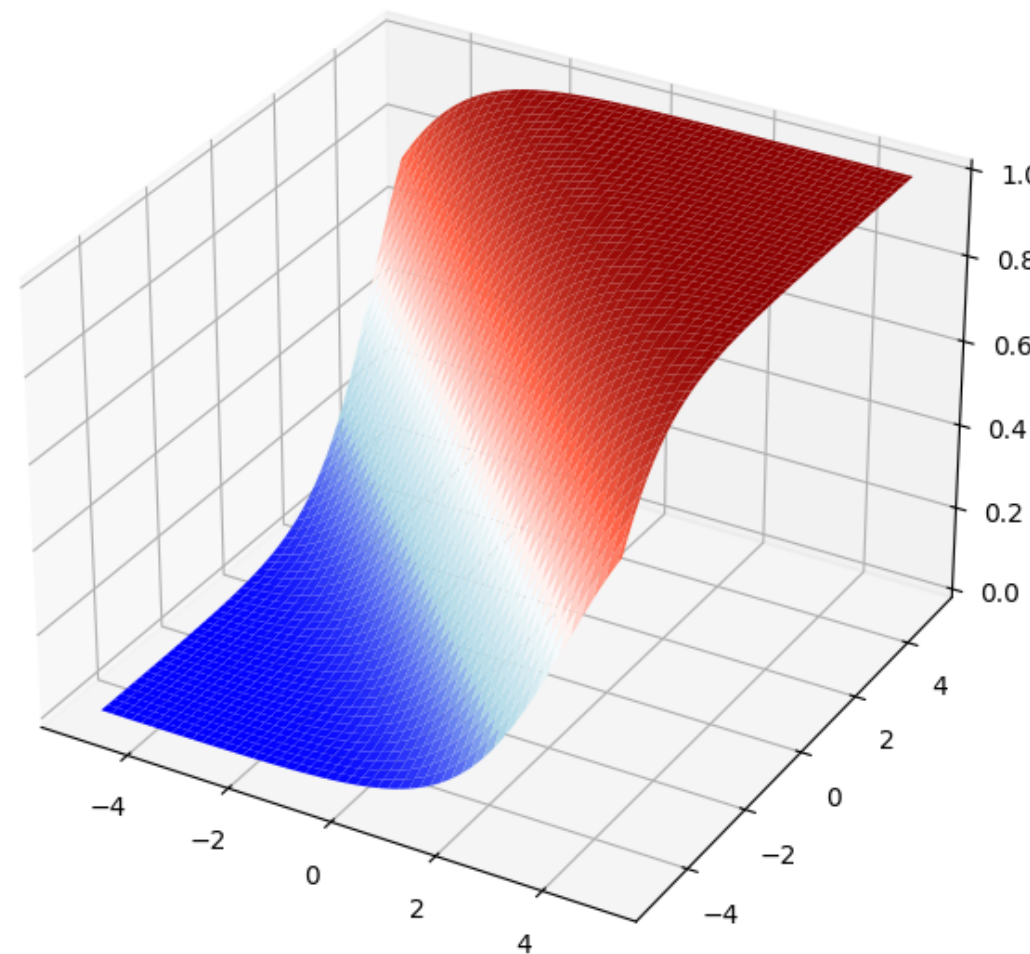


確率論的二値分類>モデル>例

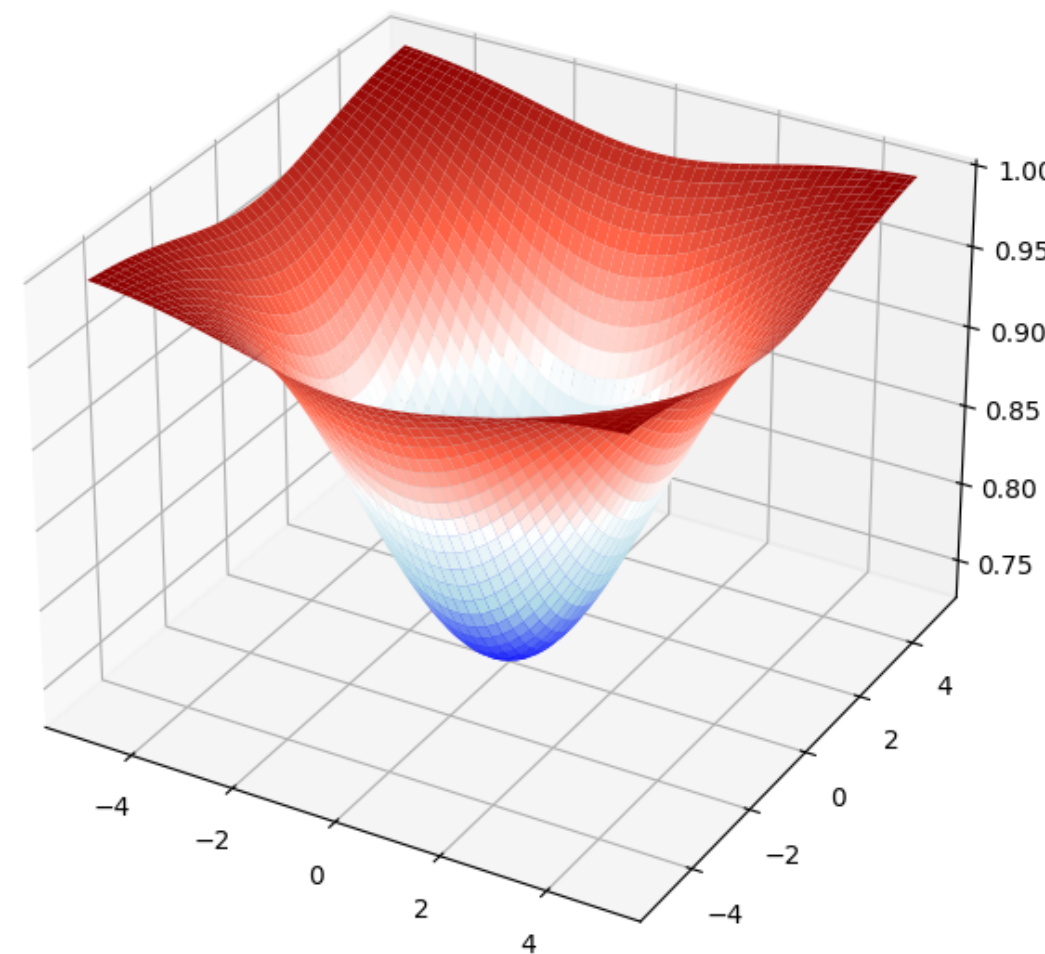
0から1までの出力に収めながらも自由なモデルを作ることが可能

■ ロジスティック関数による確率モデルの例

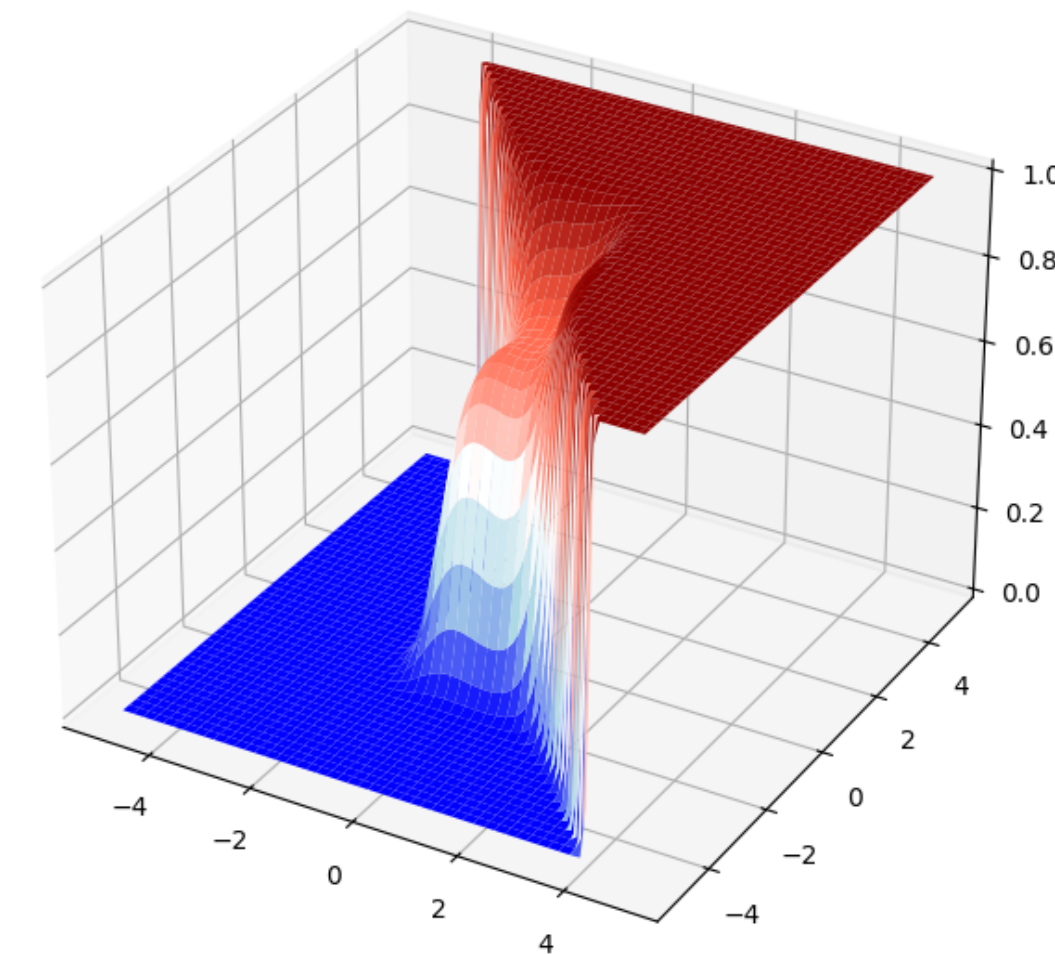
$$g(\mathbf{x}) = \sigma(1 + x_1 + x_2)$$



$$g(\mathbf{x}) = \sigma(1 + 0.1x_1^2 + 0.1x_2^2)$$



$$g(\mathbf{x}) = \sigma(1 + x_1^3 + 0.5x_2^3 + 0.2x_1x_2)$$



確率論的二値分類 > 学習法 > 正則化付き最尤法

例えばロジスティック関数によるモデルを，L2正則化付きの最尤法で学習

- ロジスティック関数による確率モデル

$$g_{\theta}(x) = \sigma(s_{\theta}(x))$$

- リスク関数として，二値交差エントロピーが小さいモデルを目指す

$$R(g_{\theta}) := \mathbb{E}[-Y \log g_{\theta}(X) - (1 - Y) \log(1 - g_{\theta}(X))]$$

- そのために，目的関数を経験リスク + L2正則化項で定義し，これを最小化

$$L(\theta) := \frac{1}{n} \sum_{i=1}^n [-y_i \cdot \log g_{\theta}(x_i) - (1 - y_i) \cdot \log(1 - g_{\theta}(x_i))] + \lambda \|\theta\|^2$$

- L2正則化付き最尤法 (L2-regularized maximum likelihood estimation)

確率論的二値分類 > 最適化 > 微分



「微分して0とおく」方法を試すために勾配を計算する

- 線型モデルの最小二乗回帰では、一階の条件から一意な最適解を見つけられた。
- 今の場合で同じことができるか、試してみよう。
 - 損失関数は次式だった

$$\ell((\mathbf{x}, y), g_{\theta}) := -y \log g_{\theta}(\mathbf{x}) - (1 - y) \log(1 - g_{\theta}(\mathbf{x}))$$

- 目的関数は次式だった

$$L(\boldsymbol{\theta}) := \frac{1}{n} \sum_{i=1}^n \ell((\mathbf{x}_i, y_i), g_{\theta}) + \lambda \|\boldsymbol{\theta}\|^2$$

確率論的二値分類 > 最適化 > 微分

ロジスティック関数による確率モデルでの、交差エントロピー損失の微分

■ 状況設定

- モデル $g_{\theta}(\mathbf{x}) = \sigma(s_{\theta}(\mathbf{x}))$
- 損失関数 $\ell((\mathbf{x}, y), g_{\theta}) := -y \log g_{\theta}(\mathbf{x}) - (1 - y) \log(1 - g_{\theta}(\mathbf{x}))$

■ このとき、勾配は次式のようになる

$$\nabla \ell((\mathbf{x}, y), g_{\theta}) = (g_{\theta}(\mathbf{x}) - y) \nabla s_{\theta}(\mathbf{x})$$

モデルの出力値

中間出力値の勾配

確率論的二値分類 > 最適化 > 微分

勾配は次式のようになる

- 勾配は次式のようになる

$$\nabla \ell((\mathbf{x}, y), g_{\theta}) = (g_{\theta}(\mathbf{x}) - y) \nabla s_{\theta}(\mathbf{x})$$

- よって、L2正則化付き最尤法では、

$$\nabla L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (g_{\theta}(\mathbf{x}_i) - y_i) \nabla s_{\theta}(\mathbf{x}_i) + 2\lambda \boldsymbol{\theta}$$

- 例えば、 $s_{\theta}(\mathbf{x})$ が線型モデルなら、一階の条件から最適解を見つけられそうか？

確率論的二値分類＞用語＞ロジスティック回帰

線型モデル＋ロジスティック関数の，最尤法ベースの学習

■ ロジスティックモデル (logistic model)

- 線型モデル＋ロジスティック関数というモデルのこと

$$g_{\theta}(x) = \sigma(\theta^{\top} \phi(x)) = \frac{1}{1 + e^{-\theta^{\top} \phi(x)}}$$

■ ロジスティック回帰 (logistic regression)

- ロジスティックモデルの最尤推定のこと（正則化が付くこともある）

$$L(\theta) := \frac{1}{n} \sum_{i=1}^n \left[-y_i \cdot \log g_{\theta}(x_i) - (1 - y_i) \cdot \log(1 - g_{\theta}(x_i)) \right] + \lambda \|\theta\|^2$$

- 注意：ロジスティック回帰は，名前に反して分類の手法

確率論的二値分類 > ロジスティックモデル > 勾配

残念ながら、 $\nabla L(\boldsymbol{\theta}) = 0$ の解の公式はない

- ロジスティックモデルの場合、 $g_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}))$ なので、

$$\nabla L(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (\sigma(\boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}_i)) - y_i) \boldsymbol{\phi}(\mathbf{x}_i) + 2\lambda \boldsymbol{\theta}$$

- よって $\nabla L(\boldsymbol{\theta}) = \mathbf{0}$ は、次と等価.

$$\frac{1}{n} \sum_{i=1}^n (\sigma(\boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}_i)) - y_i) \boldsymbol{\phi}(\mathbf{x}_i) + 2\lambda \boldsymbol{\theta} = \mathbf{0}$$

- 残念ながら、この方程式の解の公式は知られていない

確率論的二値分類 > ロジスティックモデル > 最適化の困難



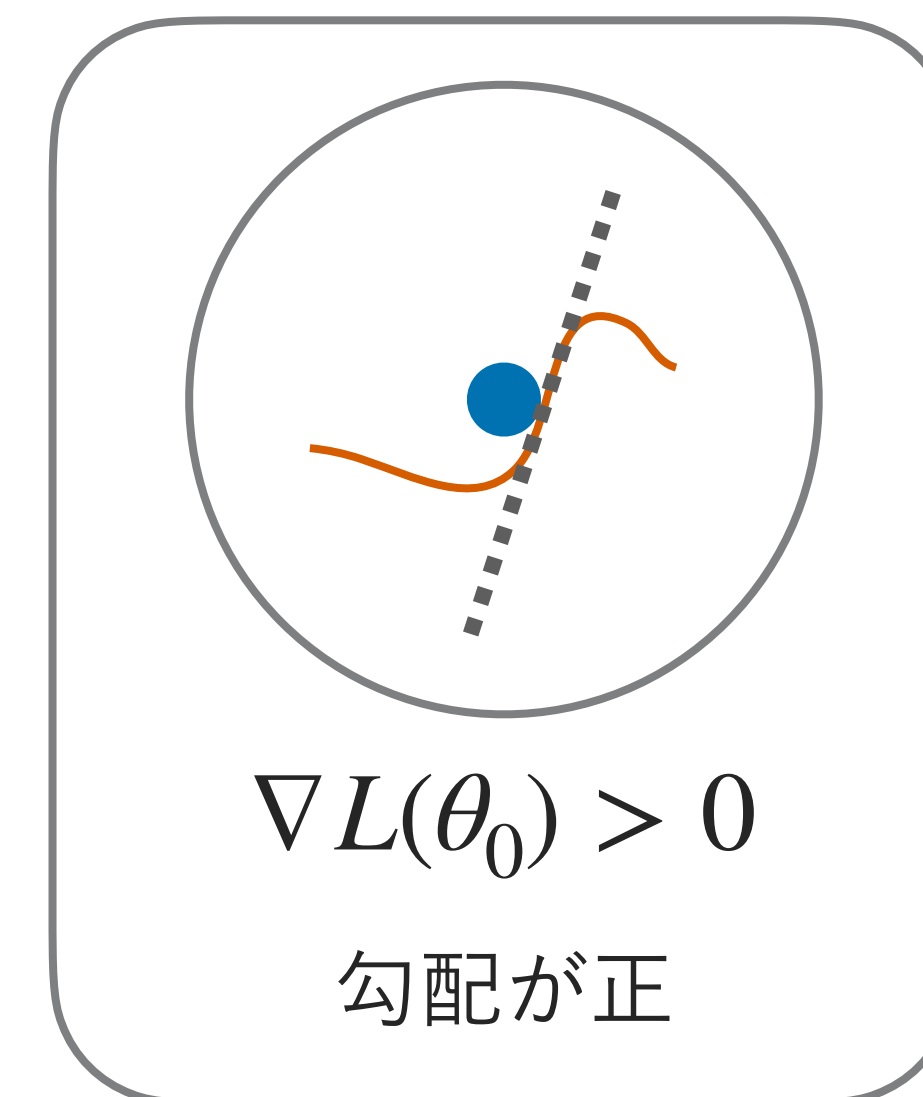
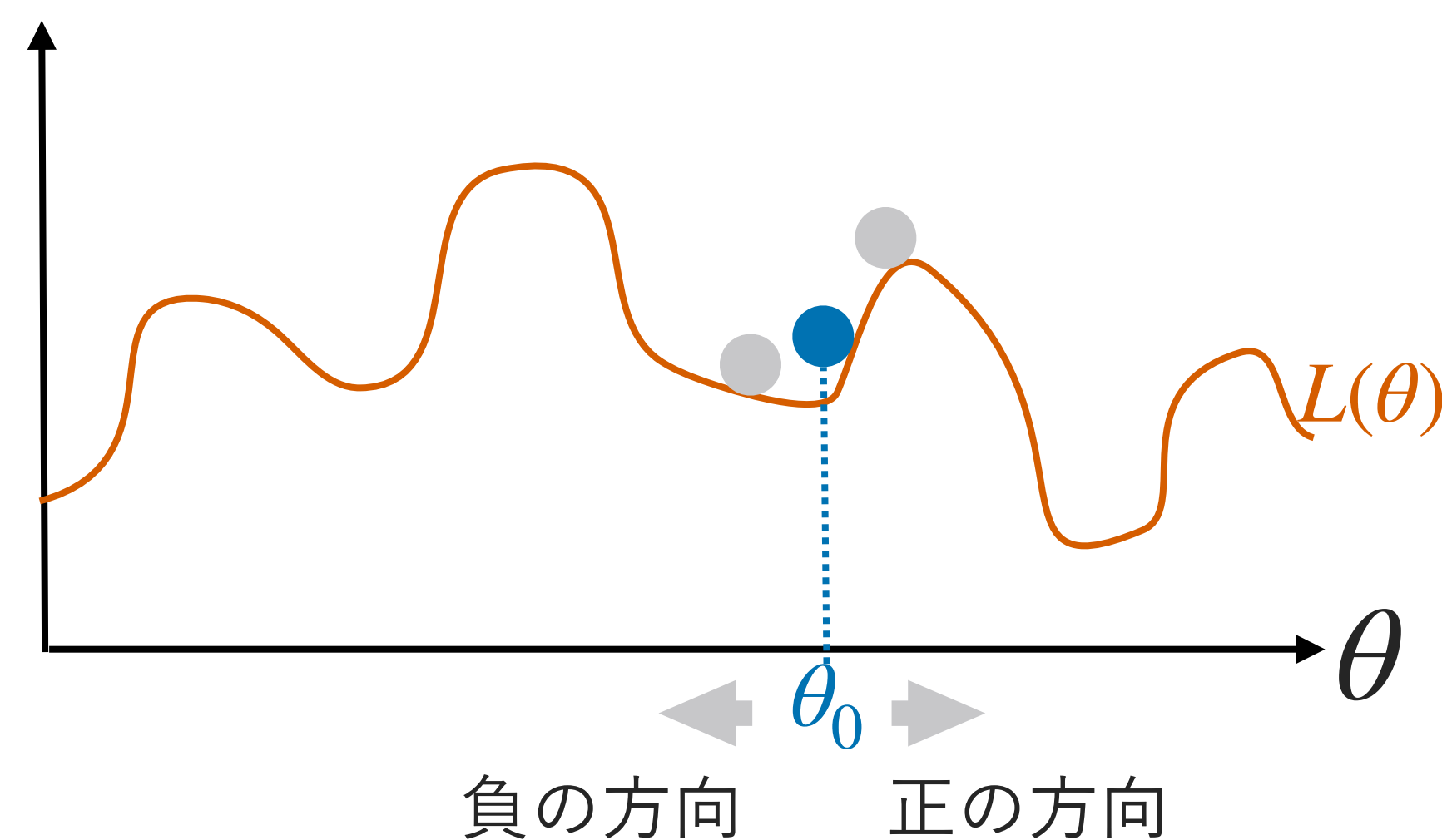
「微分して0とおく」で一意的な最適解を得るという手が使えない

- それでも、勾配を活用して最適化をする方法がある
- 勾配法 (gradient-based methods)
 - 勾配情報をもとにパラメーターを繰り返し更新するタイプの最適化手法の総称

基本知識の確認＞勾配のもつ意味

勾配は、最も上昇する方向（steepest ascent direction）を表す

- 1次元の場合、微小距離だけ θ を移動させるとき、
 - θ を $\nabla L(\theta)$ の方向に移動すれば、 L は最も増加する
 - θ を $-\nabla L(\theta)$ の方向に移動すれば、 L は最も減少する

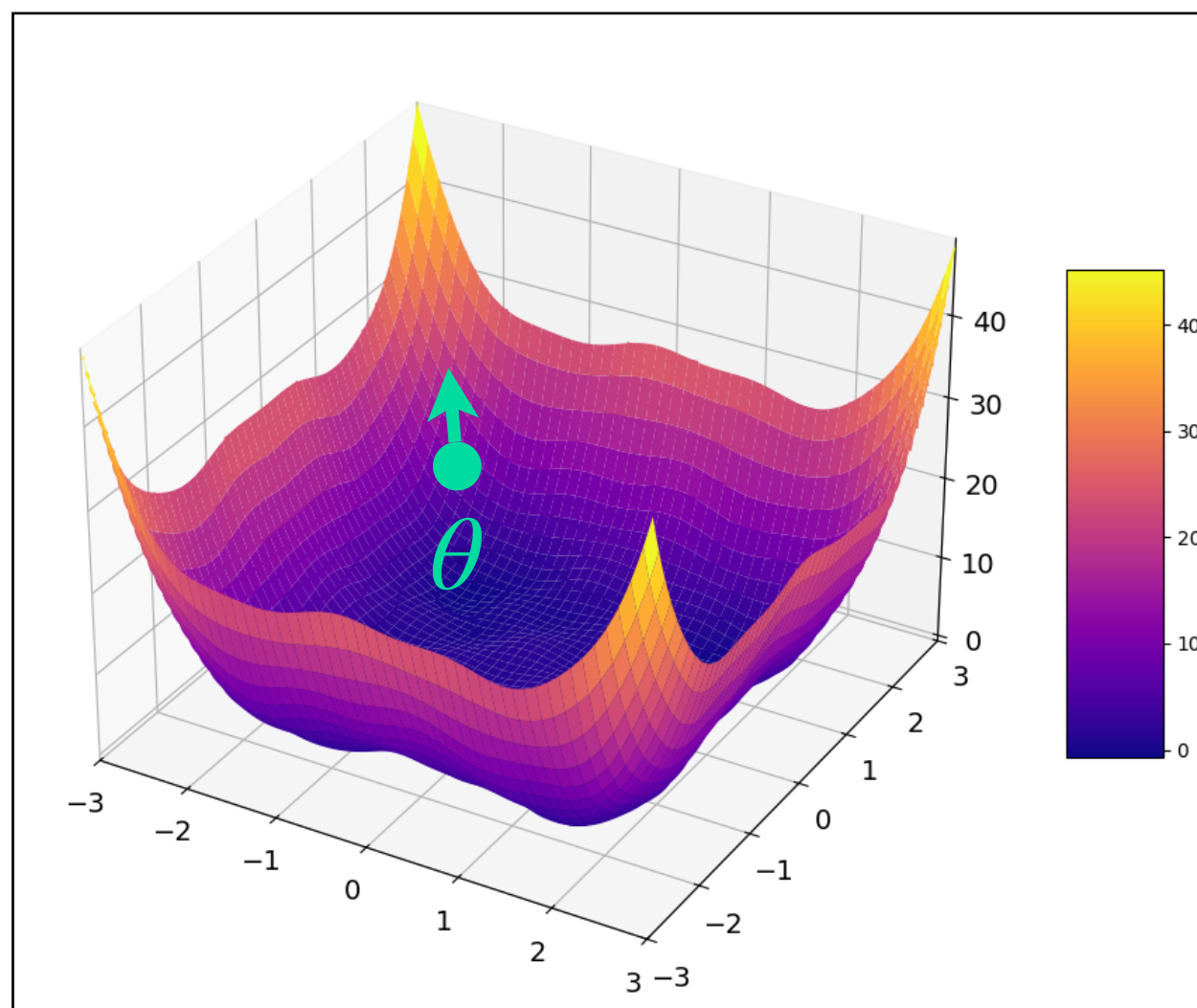


【補足】1次元の場合、勾配は単なる1変数の微分に他ならない： $\nabla L(\theta) = \frac{d}{d\theta} L(\theta)$.

基本知識の確認＞勾配のもつ意味

勾配は、最も上昇する方向（steepest ascent direction）を表す

- 多次元の場合も同じ。微小距離だけ θ を移動させるとき、
 - θ を $\nabla L(\theta)$ の方向に移動すれば、 L は最も増加する
 - θ を $-\nabla L(\theta)$ の方向に移動すれば、 L は最も減少する



勾配降下法による最適化＞アルゴリズム

勾配降下法（gradient descent）により目的関数の最小化を目指す

- パラメーター θ を適当に初期化 ($\hat{\theta} \leftarrow \theta$)
- 以下を必要なだけ繰り返す
 - 目的関数 L の現在のパラメーター $\hat{\theta}$ における勾配 $\nabla L(\hat{\theta})$ を計算
 - 勾配の反対方向にパラメーターを少し更新（学習率 $\eta > 0$ ）

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla L(\hat{\theta})$$

- 早期停止条件を満たしていたら中断
- 最終的な $\hat{\theta}$ を使用する

勾配降下法による最適化＞詳細

基本はシンプルだが、細かく見ていくと色々な項目を決める必要がある

- パラメーター θ を適当に初期化 ($\hat{\theta} \leftarrow \theta$)
 - ← 初期化の方法は色々ある
(ハイパーパラメーター)
- 以下を必要なだけ繰り返す
 - ← 繰り返しの最大回数 T は決め打ち
(ハイパーパラメーター)
 - 目的関数 L の現在のパラメーター $\hat{\theta}$ における勾配 $\nabla L(\hat{\theta})$ を計算
 - **勾配の反対方向** にパラメーターを少し更新 (学習率 $\eta > 0$)
 - ← 繰り返しの t 回目には $\eta = \eta_t$ など
徐々に小さくすることもある
(ハイパーパラメーター)
 - 早期停止条件を満たしていたら中断
 - ← 例えば検証損失の改善幅が
 S 回にわたって ϵ に満たなかったら
早期停止 (ハイパーパラメーター)
- 最終的な $\hat{\theta}$ を使用する

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla L(\hat{\theta})$$

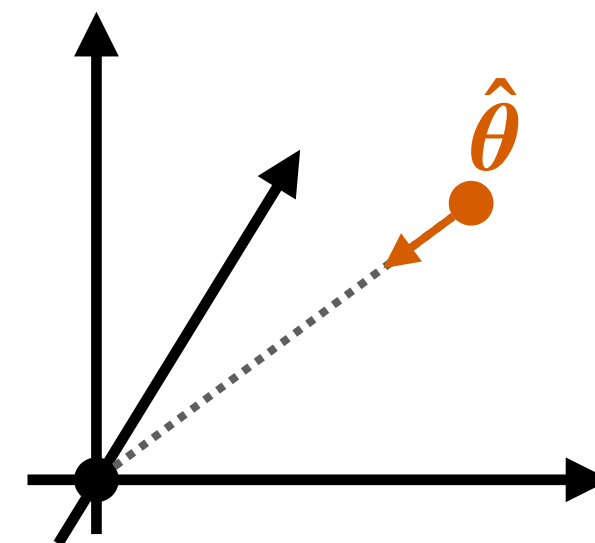
勾配降下法による最適化 > 詳細 > 重み減衰

勾配法で最適化する場合のL2正則化は，重み減衰とも呼ばれる

■ 重み減衰 (weight decay)

- 勾配法で最適化をする場合の，L2正則化の呼称
- L2正則化項があると，勾配法の更新方向は $-\nabla L(\hat{\theta}) = -\nabla \hat{R}(g_{\hat{\theta}}) - 2\lambda\hat{\theta}$
- 単に $R(g_{\theta})$ を小さくするだけの方向と比べて， $-2\lambda\hat{\theta}$ だけズレる

■ これは，更新の度に少しずつパラメーターを原点に向けて縮小すると解釈できる

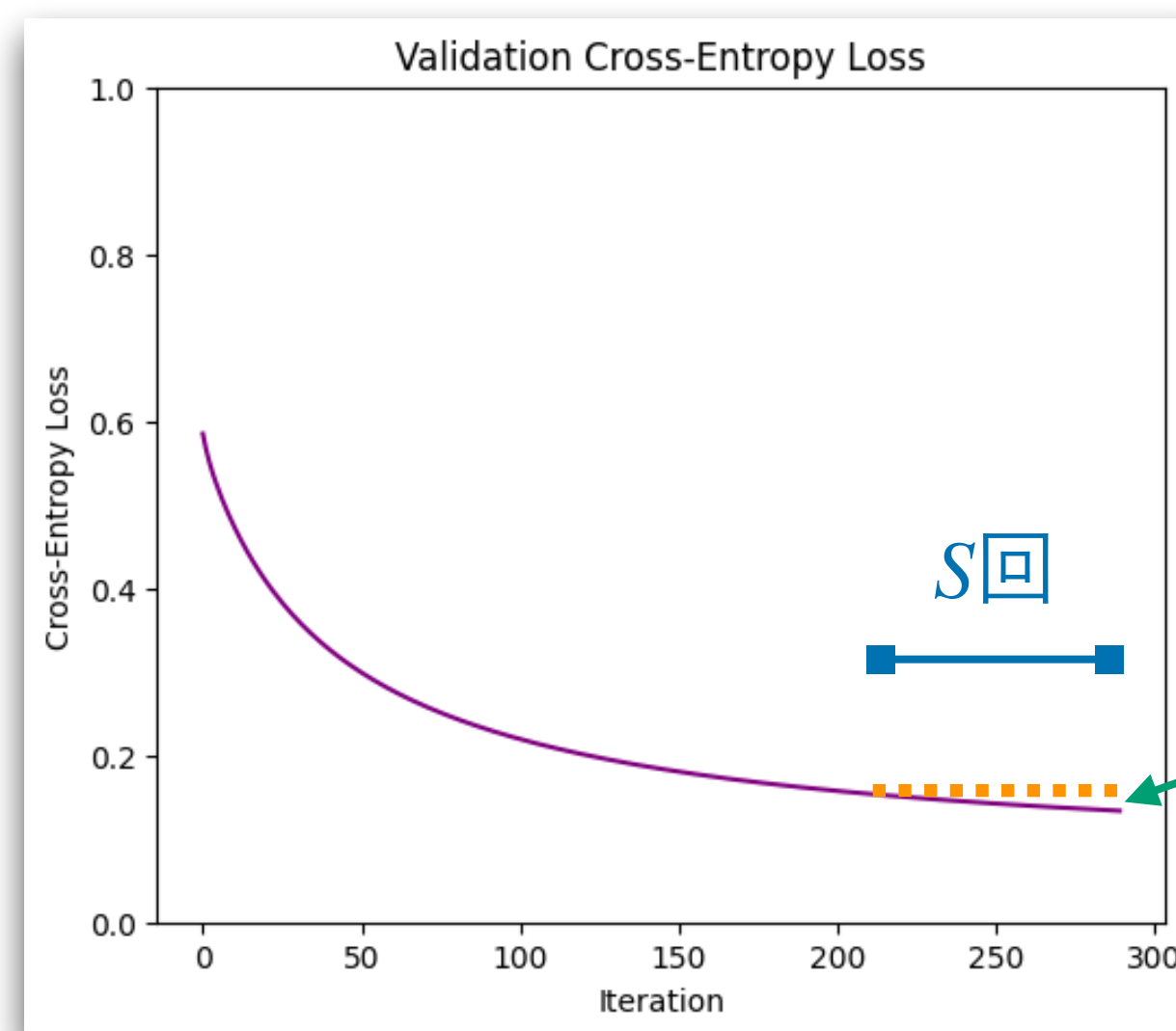


【用語】ここでの「重み」は「パラメーター」と同義（重み付き和の係数に由来する用語）。ただし，「重み」といったとき，定数項に対応するパラメーターは含まないこともある。後者の場合，定数項に対応するパラメーターはバイアスと呼ばれ，バイアスを除くパラメーターにだけ重み減衰が適用される。

勾配降下法による最適化 > 詳細 > 早期停止

何らかの指標に基づいて、早期停止（early stopping）をすることが多い

- 例）検証データ上で直近 S 回にわたって ε 以上の改善がみられなかったら終了



検証損失値

- 「更新回数」（iteration）というハイパーパラメーターの選択にほかならない

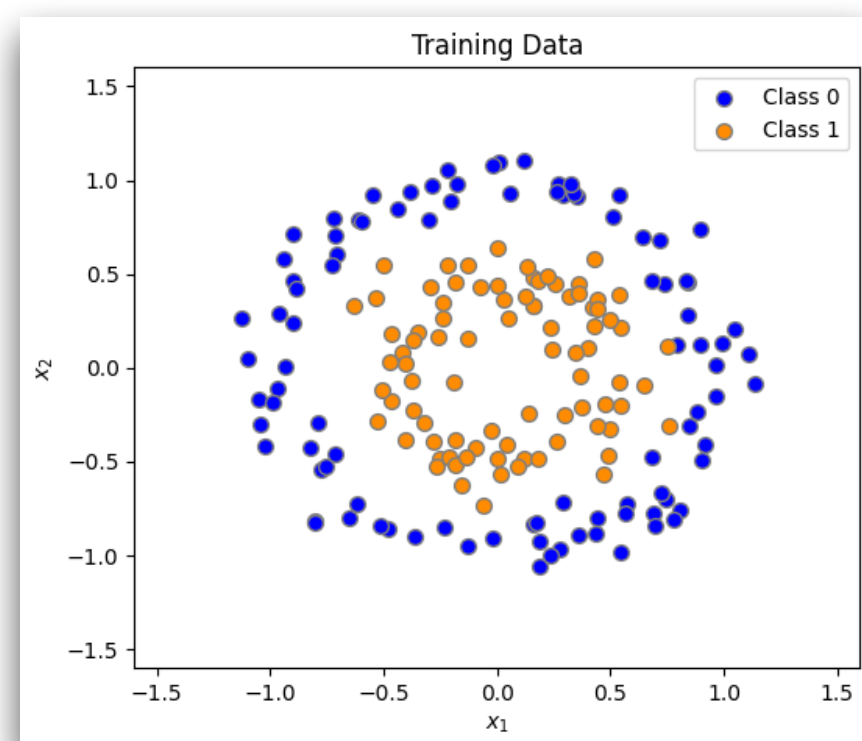
確率論的二値分類 > 実験 > 設定

ロジスティックモデルをL2正則化付き最尤法で学習（勾配降下法で最適化）

タスク・学習戦略

分類タスク・教師付き学習

特徴量	ラベル
x_1	y_1
x_2	y_2
$\vdots$	$\vdots$
$\{(X_i, Y_i)\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} p$	



訓練データ

モデル

ロジスティックモデル

$$g_{\theta}(x) = \sigma(\theta^{\top} \phi(x))$$

- 中身は30次の多項式

目的関数

負の対数尤度+正則化

$$L(\theta) := \frac{1}{n} \sum_{i=1}^n \ell((x_i, y_i), g_{\theta}) + \lambda \|\theta\|^2$$

- ハイパーパラメーター選択
- モデル選択指標は、早期停止時点の検証損失値
- 正則化係数 λ は $\{10^{-3}, 10^{-2}, 10^{-1}, 10^0\}$ から選択

最適化手法

勾配降下法

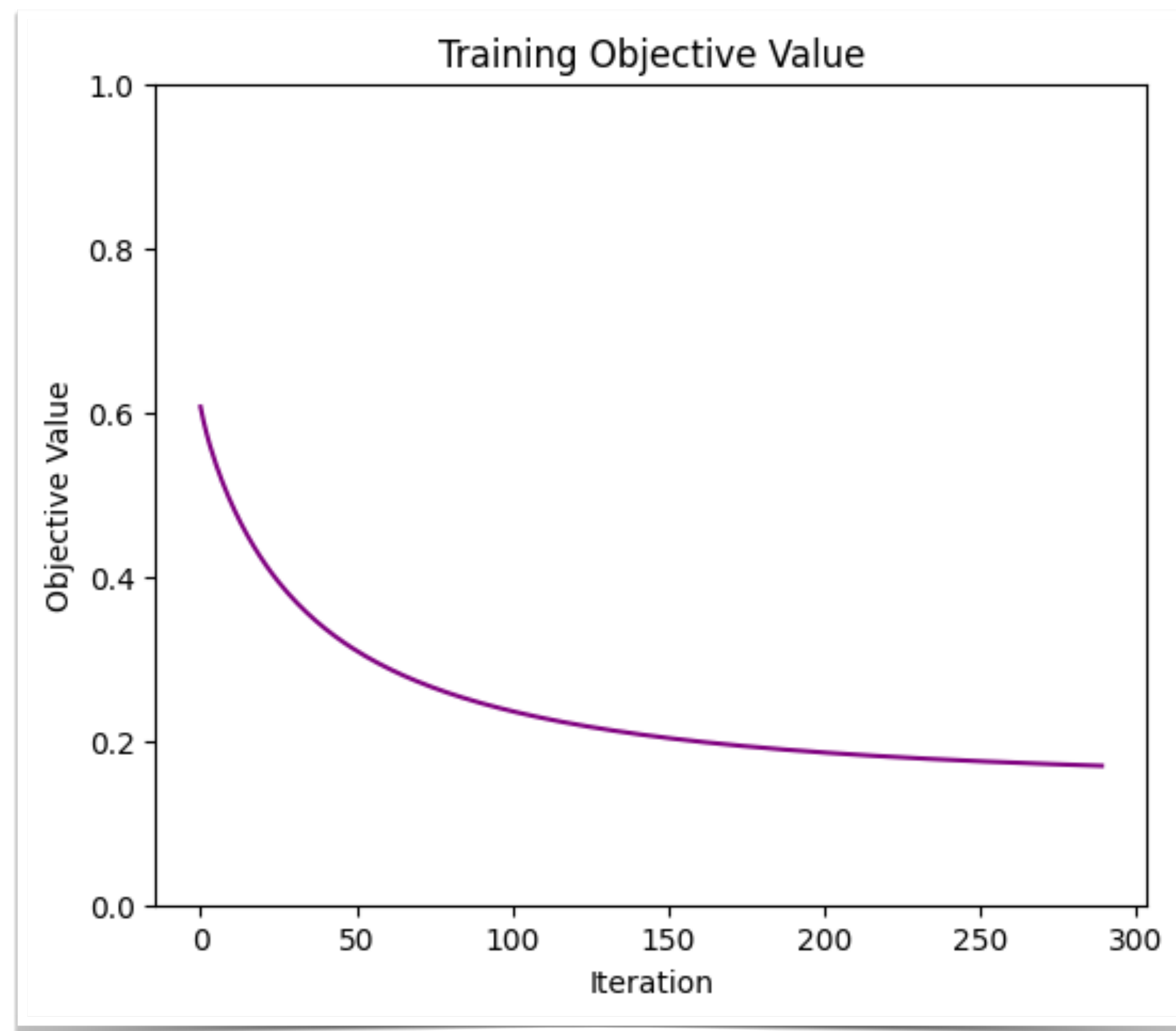
$$\hat{\theta} \leftarrow \hat{\theta} - \eta \nabla L(\hat{\theta})$$

- 学習率 η は 0.5
- 更新回数上限10000回
- 早期停止あり（直近5更新の改善幅が 10^{-3} 未満で停止）

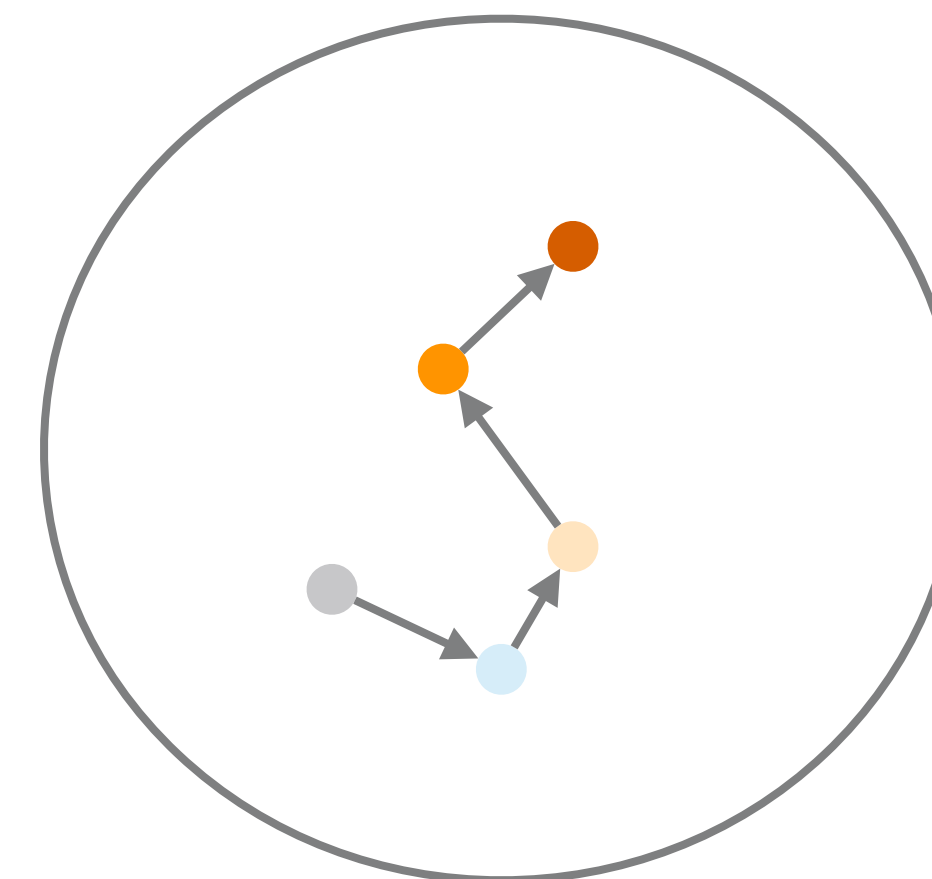
確率論的二値分類＞実験＞結果（学習過程）

更新の繰り返しにより，徐々に目的関数値が小さいパラメーターを探索

縦軸：目的関数値



横軸：更新回数



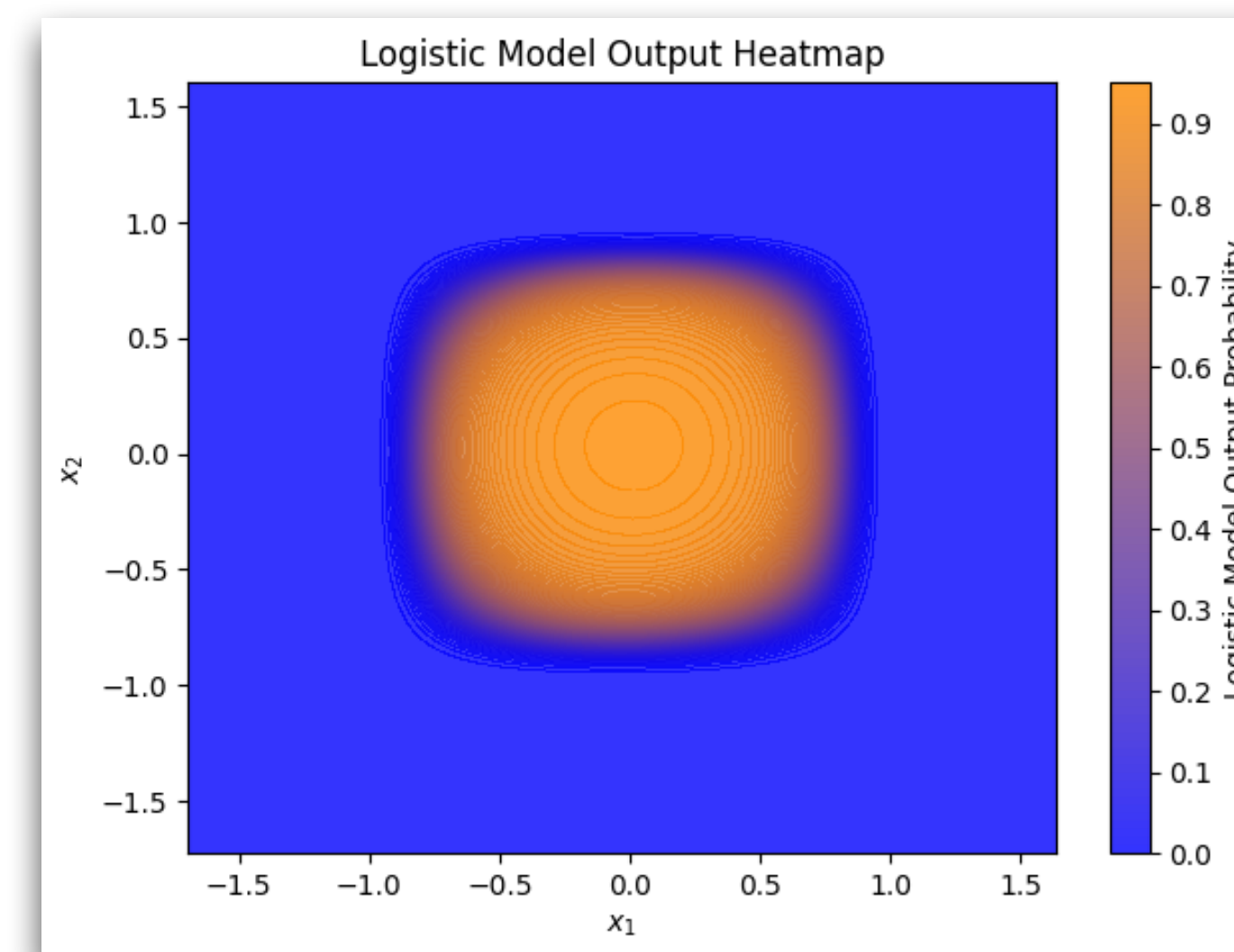
確率論的二値分類 > 実験 > 学習結果

妥当そうなスコアリングモデルが学習できている

- 正例のあたりでは 1 に近く，負例のあたりでは 0 に近いスコア
 - その境目付近ではグラデーションがついている



訓練データ

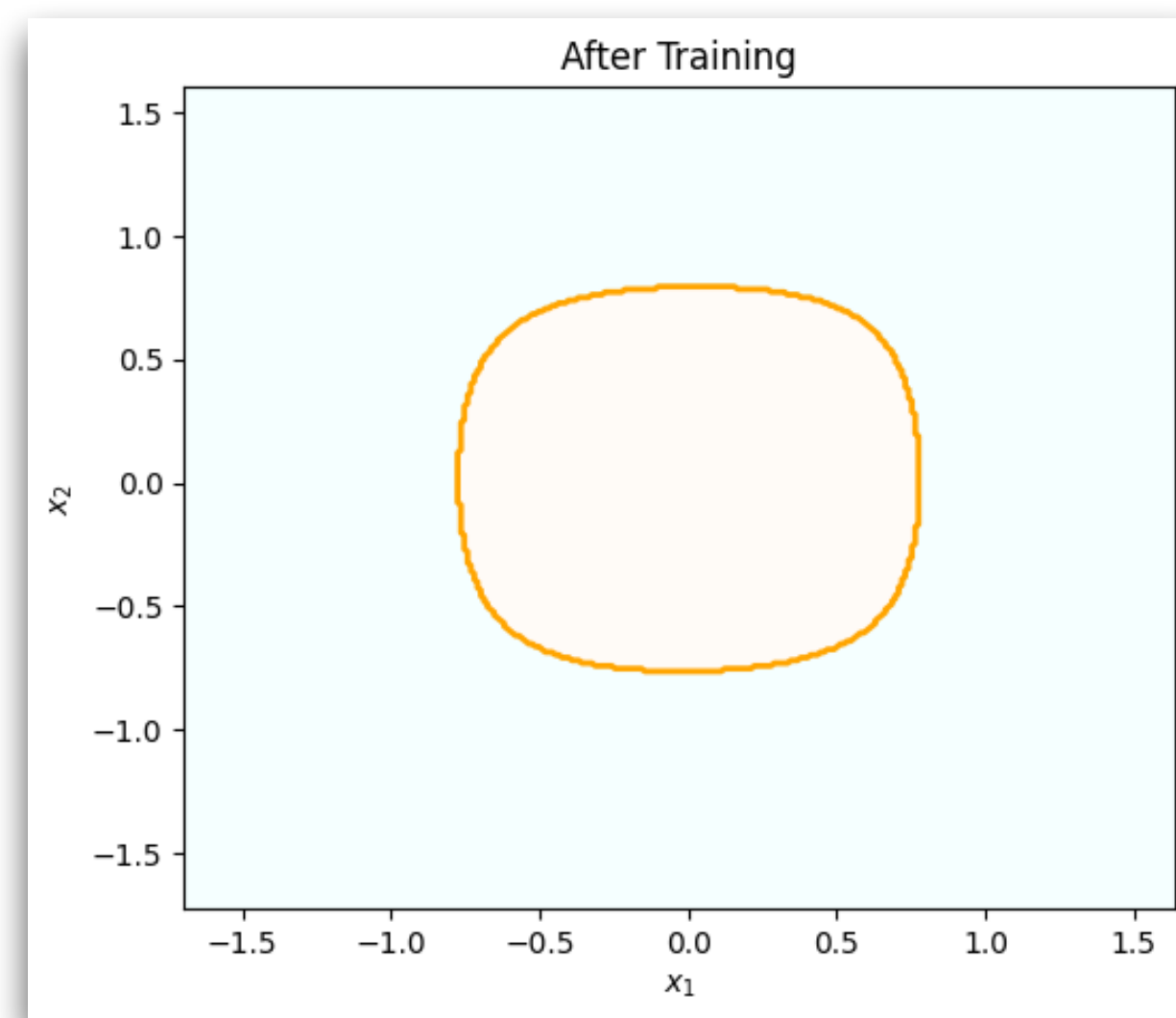


学習済みモデル $g_{\hat{\theta}}$ が出力した確率値スコア

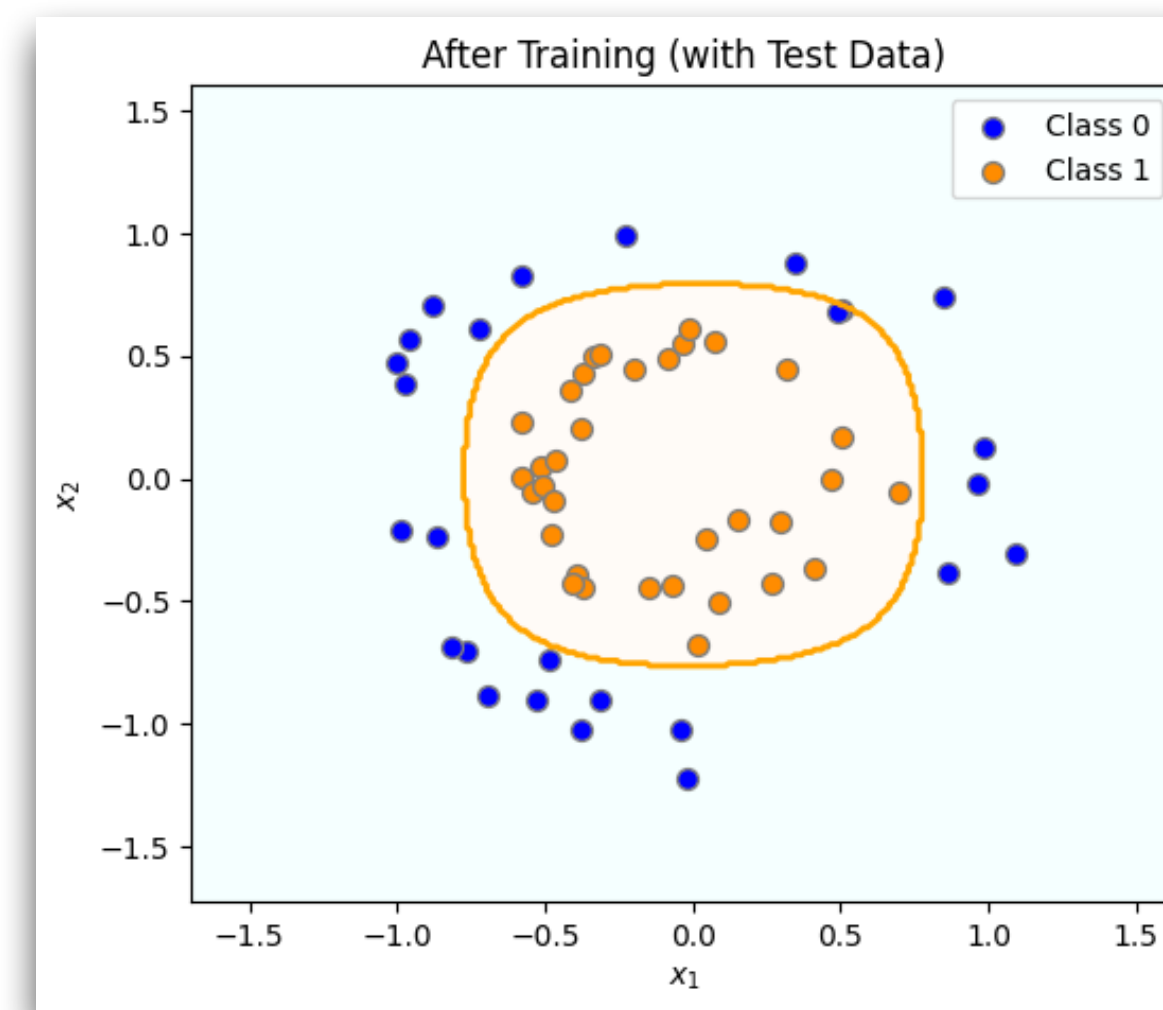
確率論的二値分類 > 実験 > 学習結果

予測器としても良さそうなものになっている

- 学習後のモデルに決定規則を合成した予測器 $f_{\hat{\theta}}(\mathbf{x}) := \tau(g_{\hat{\theta}}(\mathbf{x}))$ の様子



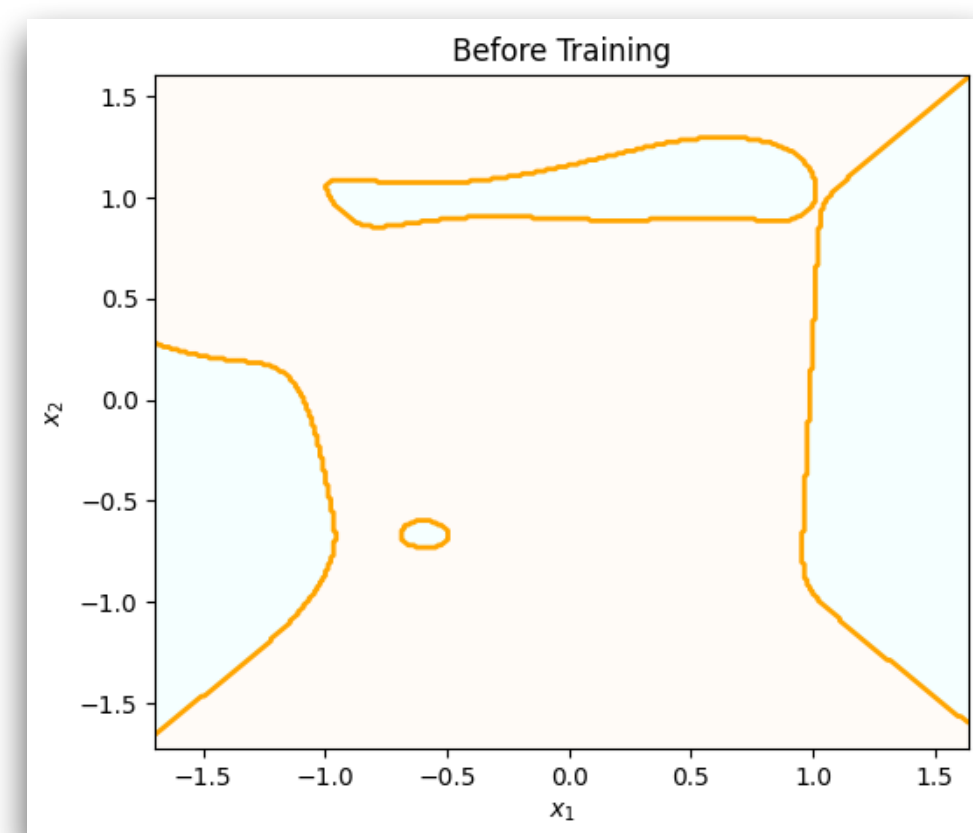
分類器としての出力
(スコアを0.5で閾値処理した結果)



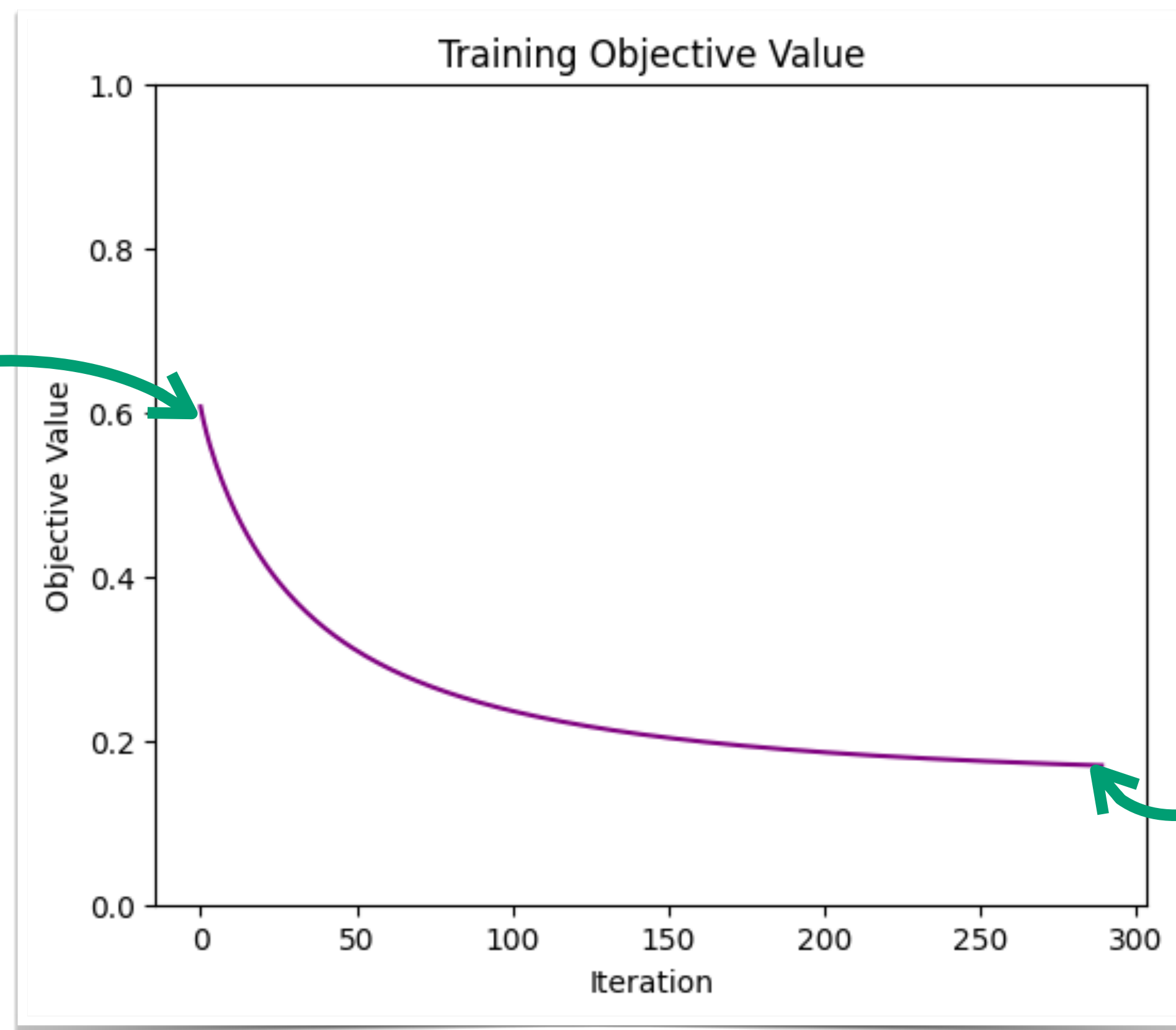
評価用データでの予測と正解ラベル

確率論的二値分類 > 実験 > 学習結果（参考）

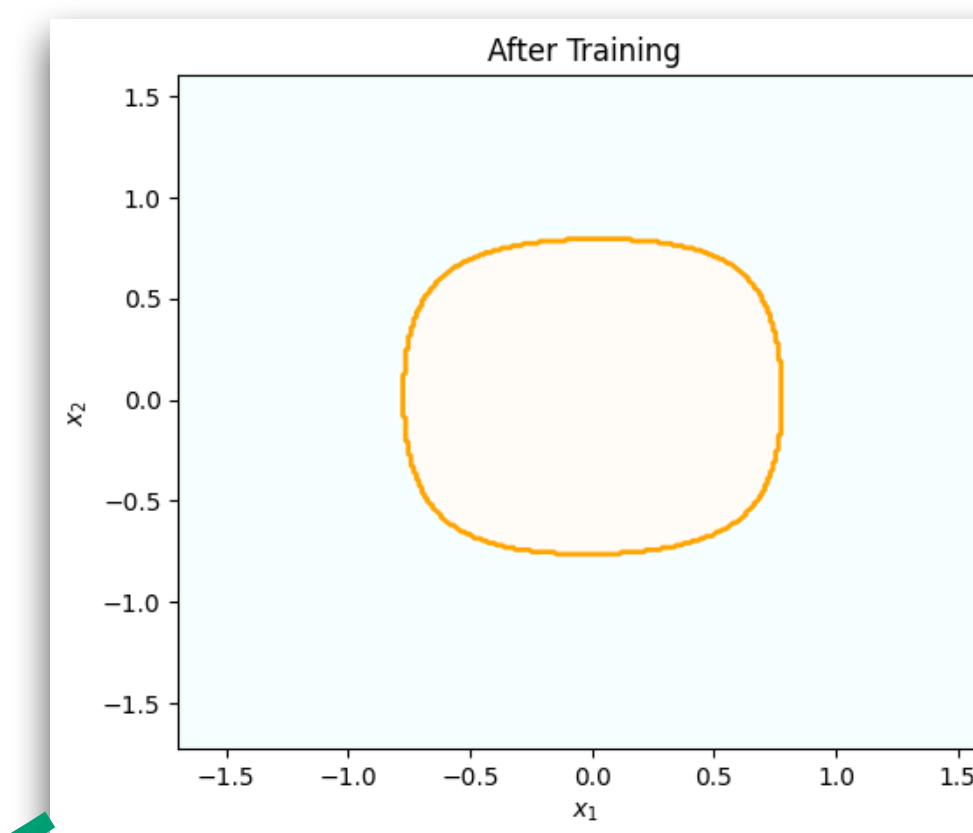
訓練前の初期化した時点での予測器と，訓練後の予測器



訓練前



目的関数値



訓練後

確率論的二値分類 > まとめ



確率論的二値分類の標準的な枠組みを説明した

■ 確率論的二値分類

- ロジスティック関数を組み込むことでモデル $g_{\theta}(\mathbf{x})$ の出力値が確率値になるようにする
- 交差エントロピーをリスクとして学習することで $g_{\theta}(\mathbf{x}) \simeq p(y = 1 | \mathbf{x})$ となるようにする
- 最適化は勾配法ベースで行う

■ 注意

- 確率値を出力できるなら、ロジスティック関数でなくてもよい
- 確率値の近似をさせられるなら、交差エントロピーの最小化でなくてもよい
- 上手く最適化できるなら、勾配法でなくてもよい
- ただし、現在はこの組み合わせが標準的（特に工夫しない場合に使われる）

確率論的二値分類 > 補足 > 勾配の導出

勾配の式 $\nabla \ell((\mathbf{x}, y), g_\theta) = (g_\theta(\mathbf{x}) - y) \nabla s_\theta(\mathbf{x})$ の導出

■ $\frac{d}{dz} \sigma(z) = \sigma(z)^2 \cdot e^{-z}$ と $\sigma(z)e^{-z} = 1 - \sigma(z)$ より, ① $\frac{d}{dz} \log \sigma(z) = 1 - \sigma(z)$ を示せる.

● [証明] $\frac{d}{dz} \log \sigma(z) = (\sigma(z))^{-1} \cdot \left(\frac{d}{dz} \sigma(z) \right) = \sigma(z) \cdot e^{-z} = 1 - \sigma(z).$

■ $\tilde{y} := 2y - 1$ と定めると, ② $\ell((\mathbf{x}, y), g_\theta) = (-\log \sigma(s_\theta(\mathbf{x})\tilde{y}))$ を示せる.

● [証明] $1 - \sigma(z) = 1 - (1 + e^{-z})^{-1} = (1 + e^{-z} - 1)(1 + e^{-z})^{-1} = (1 + e^z) = \sigma(-z).$

■ ①と②から, $\nabla \ell((\mathbf{x}, y), g_\theta) = (g_\theta(\mathbf{x}) - y) \nabla s_\theta(\mathbf{x})$ を得る.

$$\nabla \ell((\mathbf{x}, y), g_\theta) = - (1 - \sigma(s_\theta(\mathbf{x})\tilde{y})) \cdot (\nabla s_\theta(\mathbf{x})\tilde{y})$$

● [証明]
$$= \begin{cases} (\sigma(s_\theta(\mathbf{x})) - 1) \nabla s_\theta(\mathbf{x}) & (y = 1) \\ (1 - \sigma(-s_\theta(\mathbf{x}))) \nabla s_\theta(\mathbf{x}) & (y = 0) \end{cases} = \begin{cases} (\sigma(s_\theta(\mathbf{x})) - 1) \nabla s_\theta(\mathbf{x}) & (y = 1) \\ (\sigma(s_\theta(\mathbf{x}))) \nabla s_\theta(\mathbf{x}) & (y = 0) \end{cases}$$

【補足】最後の式変形は, $y = 0, 1$ をそれぞれ代入すると確認できる.

ロードマップ

